



Uniwersytet
Kaliski

im. Prezydenta Stanisława Wojciechowskiego

Projekt: MagistralaCAN4

Analizator magistrali CAN z silnikiem uczenia asocjacyjnego



MagistralaCAN4 v2.1.0

CAN Bus Analyzer · Qt6 · C++17 · Windows / Linux

PCAN · SLCAN · ESP32 · GVRET · WebSocket Remote CAN

Wykonał: Dominik Maślak

Nr indeksu: 27489

Kierunek i tryb: Elektrotechnika, niestacjonarne

Semestr: 2

Prowadzący: mgr inż. Jakub Konopiński

Kalisz, 2026

Streszczenie

MagistralaCAN4 (wersja 2.1.0) jest wielofunkcyjnym analizatorem magistrali CAN przeznaczonym dla inżynierów elektroniki pojazdowej i systemów wbudowanych. Aplikacja łączy w jednym oknie Qt6 funkcje sniffingu CAN w czasie rzeczywistym, dekodowania kilkunastu protokołów warstwy aplikacyjnej (UDS/ISO-TP, J1939, OBD-II, CANopen, SOME/IP, DoIP, KWP2000, XCP, LIN), uczenia asocjacyjnego bez etykiet oraz zdalnego dostępu przez sieć WebSocket (`ws://` / `wss://`).

Projekt obsługuje cztery klasy interfejsów sprzętowych: PCAN-USB (Peak Systems), SLCAN (Lawicel ASCII), ESP32+MCP2515 (własny firmware UART) oraz GVRET (protokół binarny EVTV — kompatybilny z SavvyCAN). Silnik uczenia maszynowego (**LearningEngine**) implementuje 37 algorytmów — od korelacji Pearsona i przyczynowości Grangera, przez k -średnich i PCA, po adaptacyjne wykrywanie anomalii EWMA.

Architektura oprogramowania opiera się na wzorcu sygnał-slot Qt z lock-free ring bufferem po stronie odbioru i timerze batch (33 ms) po stronie GUI, co pozwala na przechwytywanie do 10 000 ramek/s bez zamrożenia interfejsu. Projekt zawiera 344 testy jednostkowe (Google Test 1.14) i potok CI/CD (GitHub Actions) obejmujący budowanie na Linux i Windows, AddressSanitizer, pomiar pokrycia kodu i analizę statyczną clang-tidy.

Słowa kluczowe: magistrala CAN, Qt6, C++17, uczenie asocjacyjne, UDS, J1939, diagnostyka pojazdów, WebSocket, PCAN, SLCAN, ESP32.

Spis treści

Streszczenie	i
1 Tytuł i zakres projektu	1
2 Cel i zakres pracy	1
3 Uzasadnienie wyboru technologii	1
4 Podstawy teoretyczne magistrali CAN	2
4.1 Protokół CAN — warstwa fizyczna i łącza danych	2
4.2 Struktura ramki CAN 2.0B	3
4.3 Mechanizm arbitrażu CSMA/CR	3
4.4 CAN FD (Flexible Data-rate)	4
5 Założenia koncepcyjne projektu	4
5.1 Idea działania	4
5.2 Główne zasady projektowe	5
6 Założenia projektowe układu	5
6.1 Wymagania sprzętowe	5
6.2 Wymagania programowe	6
7 Schemat blokowy urządzenia	6
7.1 Architektura warstwowa	6
7.2 Schemat blokowy warstwy analizy	7
7.3 Schemat interfejsu sprzętowego	9
8 Szczegółowy opis działania urządzenia	9
8.1 Przechwytywanie ramek CAN	9
8.2 Model danych — CanFrameModel	9
8.3 Przepływ sygnałów w Qt	10
8.4 Interfejs użytkownika — grupy zakładek	10
8.5 Uczenie asocjacyjne — silnik ML	11
8.6 Silnik skryptowy Lua	13
8.7 Serwer WebSocket i REST API	14
8.8 Zdalny dostęp przez internet (WSS + token)	15
8.9 Mostek MQTT i integracje IoT	16
8.10 Baza danych ramek (SQLite + PCAP)	16
8.11 Pipeline alertów (CanAlertEngine)	17
8.12 CAN XL, CAN FD i CANopen	17
8.13 Analiza offline i odtwarzanie	18
9 Schemat blokowy algorytmu głównego	19

9.1	Opis przepływu sterowania	21
10	Opis implementacji	21
11	Analiza i symulacje	22
11.1	Testy jednostkowe	22
11.2	Weryfikacja algorytmu — przypadki testowe	23
11.2.1	Test 1: Poprawność dostępu do bajtów ramki CAN	23
11.2.2	Test 2: Dekodowanie sygnałów DBC ze skalowaniem	23
11.2.3	Test 3: Weryfikacja wydajności bufora batch	24
11.3	Wydajność bufora batch	24
11.4	Silnik uczenia asocjacyjnego — algorytm	25
11.5	Zaawansowane algorytmy ML — LearningEngine v2.1	26
11.6	Wizualizacja osi czasu (Timeline)	27
11.7	Zapewnienie jakości — testy i CI/CD	27
12	Schematy ideowe	28
12.1	Diagram sekwencji — połączenie z ICSim	28
12.2	Schemat blokowy sterownika SLCAN	30
12.3	Schemat protokołu GVRET (GvretDriver)	30
12.4	Schemat protokołu UDS / ISO-TP	31
12.5	Maszyna stanów sniffingu	32
13	Widoki layoutu	32
13.1	Struktura plików projektu	32
13.2	Diagram zależności modułów	32
14	Schematy montażowe	34
14.1	Podłączenie PCAN-USB	34
14.2	Podłączenie ESP32 + MCP2515	34
15	Zestawienie podzespołów i koszt	34
15.1	Lista podzespołów sprzętowych	35
15.2	Oprogramowanie — zależności (bezpłatne)	35
16	Struktura plików i konfiguracja	35
16.1	Pełna struktura katalogów projektu	36
16.2	Pliki konfiguracyjne i auto-save	37
17	Instrukcja użytkownika	37
17.1	Uruchomienie i pierwsze kroki	37
17.2	Podstawowe operacje	38
17.3	Filtrowanie wyrażeniami	38
17.4	Integracja z ICSim	38

18 Instrukcja serwisowa	39
18.1 Budowanie ze źródeł	39
18.2 Uruchamianie testów	39
18.3 Konfiguracja wirtualnej magistrali CAN (Linux)	39
18.4 Regeneracja tokena WSS	40
18.5 Możliwe przyczyny awarii i diagnoza	41
18.6 Pliki konfiguracyjne i logi	42
19 Wykaz skrótów i pojęć	43
20 Uwagi dodatkowe	44
20.1 Historia wersji	44
20.2 Plany rozwoju	44
20.3 Podsumowanie	45

Spis rysunków

1	Warstwy protokołu CAN wg modelu ISO/OSI	3
2	Struktura ramki CAN 2.0B (format rozszerzony, 29-bit ID)	3
3	Koncepcja przepływu danych — od źródła CAN do modułów analizy	4
4	Architektura warstwowa systemu MagistralaCAN4	7
5	Schemat blokowy warstwy analizy — przepływ <code>CanFrame</code> do modułów	8
6	Schemat połączeń sprzętowych systemu	9
7	Diagram przepływu sygnałów Qt w <code>MainWindow</code>	10
8	Makieta interfejsu użytkownika (zakładka Ruch CAN)	11
9	Architektura integracji sieciowej MagistralaCAN4	15
10	Architektura zdalnego dostępu — PC-A (serwer) ↔ PC-B (klient)	15
11	Schemat mostka MQTT — od ramek CAN do systemów IoT	16
12	Schemat blokowy algorytmu głównej pętli sniffingu CAN (ISO 5807)	20
13	Opóźnienie wyświetlenia ramki w funkcji intensywności ruchu CAN	25
14	Algorytm uczenia asocjacyjnego w module <code>AssociativeLearner</code>	26
15	Diagram sekwencji — połączenie ICSim z MagistralaCAN4 przez <code>WebSocket</code>	29
16	Schemat blokowy sterownika SLCAN (<code>SlCanDriver</code>)	30
17	Format pakietu binarnego protokołu GVRET (ramka odbioru, <code>CMD=0x02</code>)	30
18	Schemat stosu protokołów UDS/ISO-TP	31
19	Maszyna stanów przechwytywania	32
20	Uproszczony diagram zależności głównych modułów	33
21	Schemat montażowy — podłączenie PCAN-USB do magistrali CAN	34
22	Schemat montażowy — ESP32 + MCP2515 jako interfejs CAN	34

Spis tabel

1	Porównanie technologii pod kątem wymagań projektu	2
2	Wymagania sprzętowe systemu	5
3	Zależności programowe (czas kompilacji)	6
4	Grupy zakładek i zawarte widgety	11
5	Moduły analityczne AssociativeLearner / LearningEngine	12
6	API silnika Lua — funkcje dostępne w skryptach	13
7	Porównanie interfejsów WebSocket i REST API	14
8	Endpointy REST API (HttpRestServer, port 8080)	14
9	Mechanizmy bezpieczeństwa zdalnego połączenia	15
10	Funkcje modułu CanObservationDb (SQLite WAL)	17
11	Typy reguł alertów w CanAlertEngine	17
12	Porównanie standardów CAN obsługiwanych przez MagistralaCAN4	18
13	Typy ramek protokołu CANopen dekodowane przez MagistralaCAN4	18
14	Funkcje modułu OfflineAnalyzer	19
15	Kluczowe funkcje i ich sygnatury	22
16	Zestaw testów jednostkowych projektu	23
17	Opóźnienie wyświetlenia przy różnym ruchu CAN (pomiar eksperymentalny)	25
18	Zaawansowane algorytmy ML dodane w wersji 2.1	27
19	Parametry konfiguracyjne widgetu Timeline	27
20	Pliki testów jednostkowych i liczba przypadków testowych	28
21	Etapy potoku CI/CD GitHub Actions	28
22	Polecenia protokołu GVRET obsługiwane przez GvretDriver	31
23	Organizacja plików projektu	32
24	Zestawienie podzespołów opcjonalnych (interfejsy CAN)	35
25	Zestawienie zależności programowych — licencje	35
26	Szczegółowa organizacja katalogów projektu MagistralaCAN4	36
27	Pliki konfiguracyjne, sesji i auto-save aplikacji	37
28	Podstawowe operacje użytkownika	38
29	Identyfikatory CAN symulatora ICSim	39
30	Diagnoza najczęstszych problemów	41
31	Pliki konfiguracyjne i logi aplikacji	42
32	Wykaz skrótów i pojęć używanych w dokumentacji	43
33	Historia wersji projektu MagistralaCAN4	44
34	Parametry techniczne MagistralaCAN4 v2.1.0	45

1 Tytuł i zakres projektu

Pełna nazwa: MagistralaCAN4 — wielofunkcyjny analizator magistrali CAN z silnikiem uczenia asocjacyjnego, wsparciem protokołów samochodowych i zdalnym dostępem przez WebSocket.

Wersja: 2.1.0 **Platforma docelowa:** Windows 10/11, Linux **Język:** C++17 / Qt 6.6

Licencja: prywatna

Projekt stanowi pełnoprawną stację roboczą inżyniera elektroniki pojazdowej: umożliwia przechwytywanie i dekodowanie ramek CAN w czasie rzeczywistym, naukę wzorców asocjacyjnych (bez etykiet), analizę protokołów wyższych warstw (UDS, J1939, OBD-II, CANopen, SOME/IP, DoIP, KWP2000, XCP, LIN) oraz zdalną pracę przez sieć LAN/WAN.

2 Cel i zakres pracy

Celem projektu jest stworzenie narzędzia diagnostycznego dla magistrali CAN, które w jednym oknie aplikacji łączy funkcje:

- **sniffingu CAN** — przechwytywania i wyświetlania ramek z wielu źródeł sprzętowych i programowych;
- **dekodowania protokołów** — automatycznej interpretacji ramek zgodnie ze standardami branżowymi;
- **uczenia maszynowego** — asocjacyjnego odkrywania zależności między zdarzeniami a danymi magistrali, bez konieczności posiadania gotowego modelu (DBC);
- **zdalnego dostępu** — transmisji i odbioru ramek przez WebSocket (wss:// i ws://) z dowolnej lokalizacji w sieci;
- **integracji z symulatorem ICSim** — podłączenia wirtualnej magistrali z symulatora dashboardu samochodu do analizy.

Zakres obejmuje zarówno warstwę sprzętową (sterowniki PCAN, SLCAN, ESP32-MCP2515, GVRET/SavvyCAN), jak i warstwę oprogramowania (parsery, silnik ML, interfejs graficzny Qt, REST API, MQTT, nagrywanie MDF4).

3 Uzasadnienie wyboru technologii

Wybór języka i środowiska programistycznego dla analizatora magistrali CAN został przeprowadzony na podstawie analizy wymagań inżynierskich projektu:

Tabela 1: Porównanie technologii pod kątem wymagań projektu

Kryterium	C++17 + Qt6	Python + PyQt	Wybór
Wydażność przetwarzania 10k ramek/s	Niskie opóźnienie (< 100 ms), bezpośredni dostęp do pamięci	Ograniczona przez GIL, narzut interpretera	C++
Dostęp do socket-CAN	Natywne API PF_CAN, SOCK_RAW, struktury jądra	Wrapper przez ctypes/cffi, dodatkowa warstwa	C++
Wielowątkowość	QtConcurrent, std::thread, lock-free ring buffer	threading z GIL, multiprocessing (osobne procesy)	C++
Akceleracja GPU (OpenCL)	Bezpośrednie API OpenCL C, zarządzanie buforami	pyopencl — dodatkowa zależność i narzut	C++
Rozmiar pliku wykonywalnego	~12 MB (statycznie linkowane)	Wymaga interpretera + pakiety (> 200 MB)	C++
Szybkość prototypowania	Kompilacja, statyczne typowanie	Interpretowany, dynamiczne typowanie	Python
Dostępność bibliotek ML	libtorch (PyTorch C++ API) — w planach	scikit-learn, numpy, pandas — bogaty ekosystem	Python

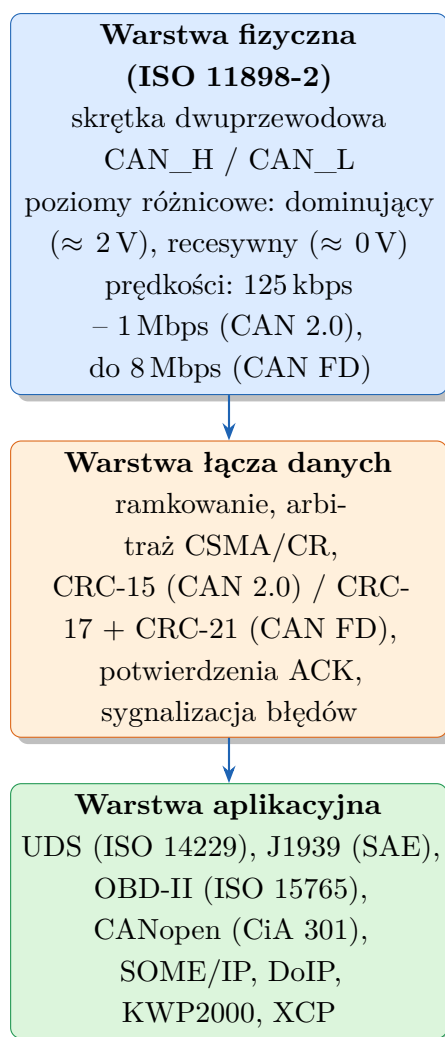
Wniosek: Dla systemu czasu rzeczywistego operującego na magistrali CAN, gdzie kluczowe są: determinizm czasowy, bezpośredni dostęp do sprzętu sieciowego jądra Linux oraz minimalne opóźnienia, język C++17 z frameworkiem Qt 6.6 stanowi optymalny wybór. Python byłby odpowiedni dla analizy offline i prototypowania algorytmów ML, jednak nie spełnia wymagań wydajnościowych dla przechwytywania na żywo przy obciążeniu > 1000 ramek/s.

Dodatkowym argumentem jest możliwość kompilacji do pojedynczego pliku wykonywalnego (static linking Qt6), co eliminuje problemy z zależnościami na komputerze docelowym — istotne w środowisku warsztatowym, gdzie instalacja interpretera Pythona i dziesiątek pakietów byłaby niepraktyczna.

4 Podstawy teoretyczne magistrali CAN

4.1 Protokół CAN — warstwa fizyczna i łącza danych

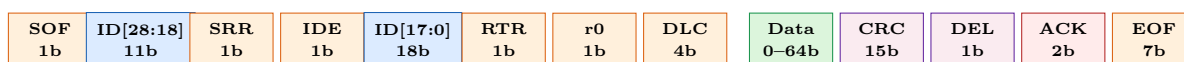
Controller Area Network (CAN) to szeregową magistrala komunikacyjna opracowana przez firmę Bosch w 1983 roku, znormalizowana jako ISO 11898. Jest podstawowym standardem komunikacji w elektronice pojazdowej, automatyce przemysłowej i systemach wbudowanych.



Rysunek 1: Warstwy protokołu CAN wg modelu ISO/OSI

4.2 Struktura ramki CAN 2.0B

Ramka standardowa CAN 2.0B (29-bitowy identyfikator) ma następującą budowę:



29-bit ID (łącznie)

Rysunek 2: Struktura ramki CAN 2.0B (format rozszerzony, 29-bit ID)

4.3 Mechanizm arbitrażu CSMA/CR

CAN wykorzystuje nieniszczący arbitraż bitowy CSMA/CR (Carrier Sense Multiple Access / Collision Resolution):

- Każdy węzeł nasłuchuje magistrali przed rozpoczęciem transmisji (carrier sense).
- Jeśli magistrala jest wolna, węzeł rozpoczyna nadawanie bitu SOF.
- Podczas nadawania pola identyfikatora węzeł jednocześnie odczytuje stan magistrali. Jeśli nadaje bit recesywny (1), a odczyta dominujący (0), oznacza to, że inny węzeł

nadaje ramkę o wyższym priorytecie (niższa wartość ID → wyższy priorytet). Węzeł przegrywający natychmiast przestaje nadawać i przechodzi w tryb odbioru.

- Mechanizm ten gwarantuje, że ramka o najniższym ID zawsze wygrywa arbitraż bez utraty danych (nieniszczący).

4.4 CAN FD (Flexible Data-rate)

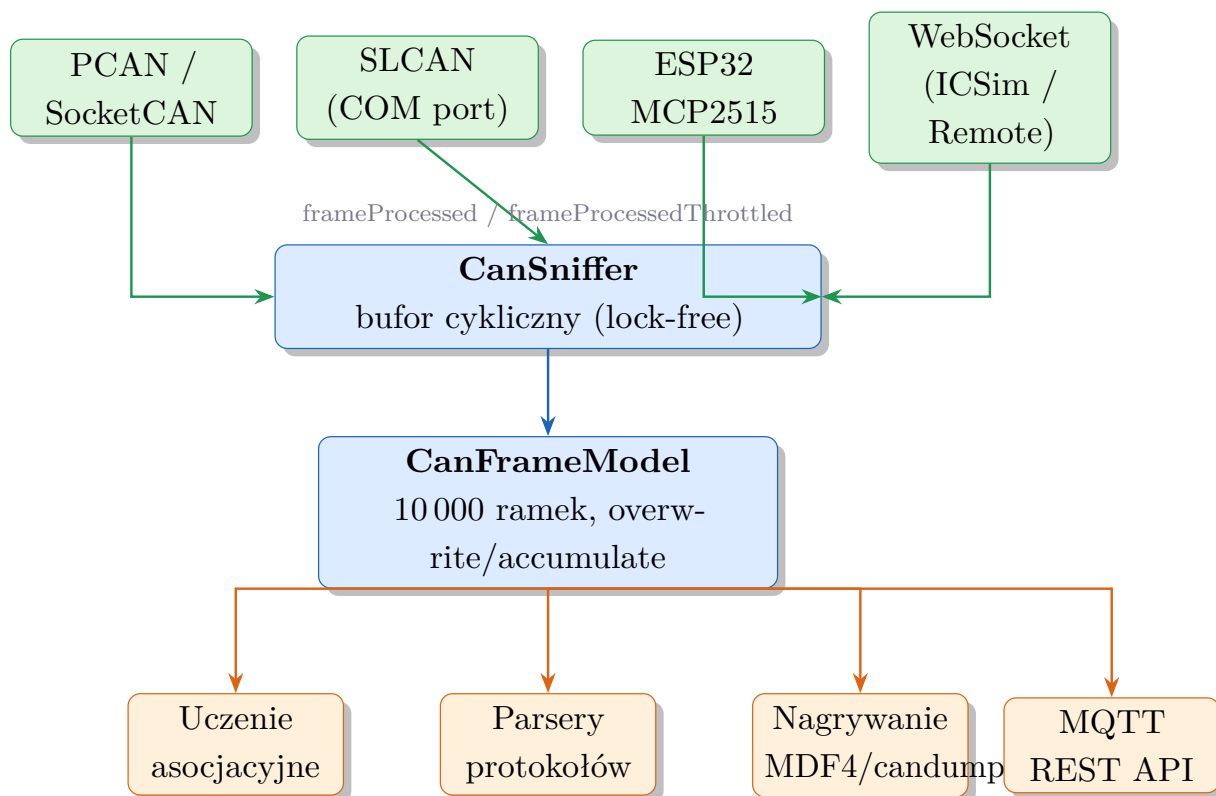
CAN FD (ISO 11898-1:2015) rozszerza klasyczny CAN o:

- **Zwiększoną przepływność fazy danych** — do 8 Mbps (przy zachowaniu standardowej prędkości fazy arbitrażu, np. 500 kbps).
- **Wydłużone pole danych** — do 64 bajtów (zamiast 8).
- **Ulepszony CRC** — CRC-17 dla ramek do 16 B, CRC-21 dla ramek 20–64 B.
- **Bit BRS** (Bit Rate Switch) — sygnalizuje przełączenie na wyższą prędkość w fazie danych.

5 Założenia koncepcyjne projektu

5.1 Idea działania

Program pełni rolę pośrednika między magistralą CAN a analitykiem. Każda ramka CAN przychodzi z jednego ze źródeł (sprzęt, plik, WebSocket), trafia do centralnego bufora i jest rozgłaszana do wszystkich aktywnych modułów analizy. Rysunek 3 ilustruje tę zasadę.



Rysunek 3: Koncepcja przepływu danych — od źródła CAN do modułów analizy

5.2 Główne zasady projektowe

1. **Pojedynczy bufor wejściowy** — wszystkie źródła wstrzykują ramki przez `MainWindow::onNewFrame()`, co eliminuje duplikaty i upraszcza synchronizację.
2. **Timer batch 33 ms** — odświeżanie tabeli i rozsyłanie sygnałów odbywa się w partii maksymalnie 500 ramek na tik, co chroni GUI przed zamrożeniem przy wysokim ruchu (>1000 ramek/s).
3. **Throttling slotów ciężkich** — widgety wizualne (dashboard, heatmap, trend) odbierają co $N = 16$ ramkę, zapewniając ok. 30 fps wizualnych bez obciążenia CPU.
4. **Lazy-loading** zakładek — parsery UDS, OBD-II, SOME/IP i inne tworzone są dopiero przy pierwszym otwarciu zakładki.

6 Założenia projektowe układu

6.1 Wymagania sprzętowe

Tabela 2: Wymagania sprzętowe systemu

Komponent	Minimalne wymaganie	Obsługiwany wariant
Interfejs CAN Magistrala CAN	USB-CAN adapter 2-przewodowa	PCAN-USB, Canable, USBtin, Peak PCAN CAN 2.0A (11-bit), CAN 2.0B (29-bit), CAN FD
Serial/SLCAN	Port COM / USB	Lawicel SLCAN, Canable, CAN232
ESP32+MCP2515	UART 115200 baud	Własny firmware ASCII sniffer
GVRET / Savvy-CAN	UART 115200 baud	Protokół binarny EVTV
Sieć LAN/WAN	TCP/IP	WebSocket ws:// (plaintext) i wss:// (TLS)
CPU	x86-64, ≥ 2 rdzenie	Windows 10/11, Linux (Ubuntu 22.04+)
RAM	512 MB	Zalecane 2 GB dla dużych sesji
Dysk	200 MB	Instalacja + logi candump

6.2 Wymagania programowe

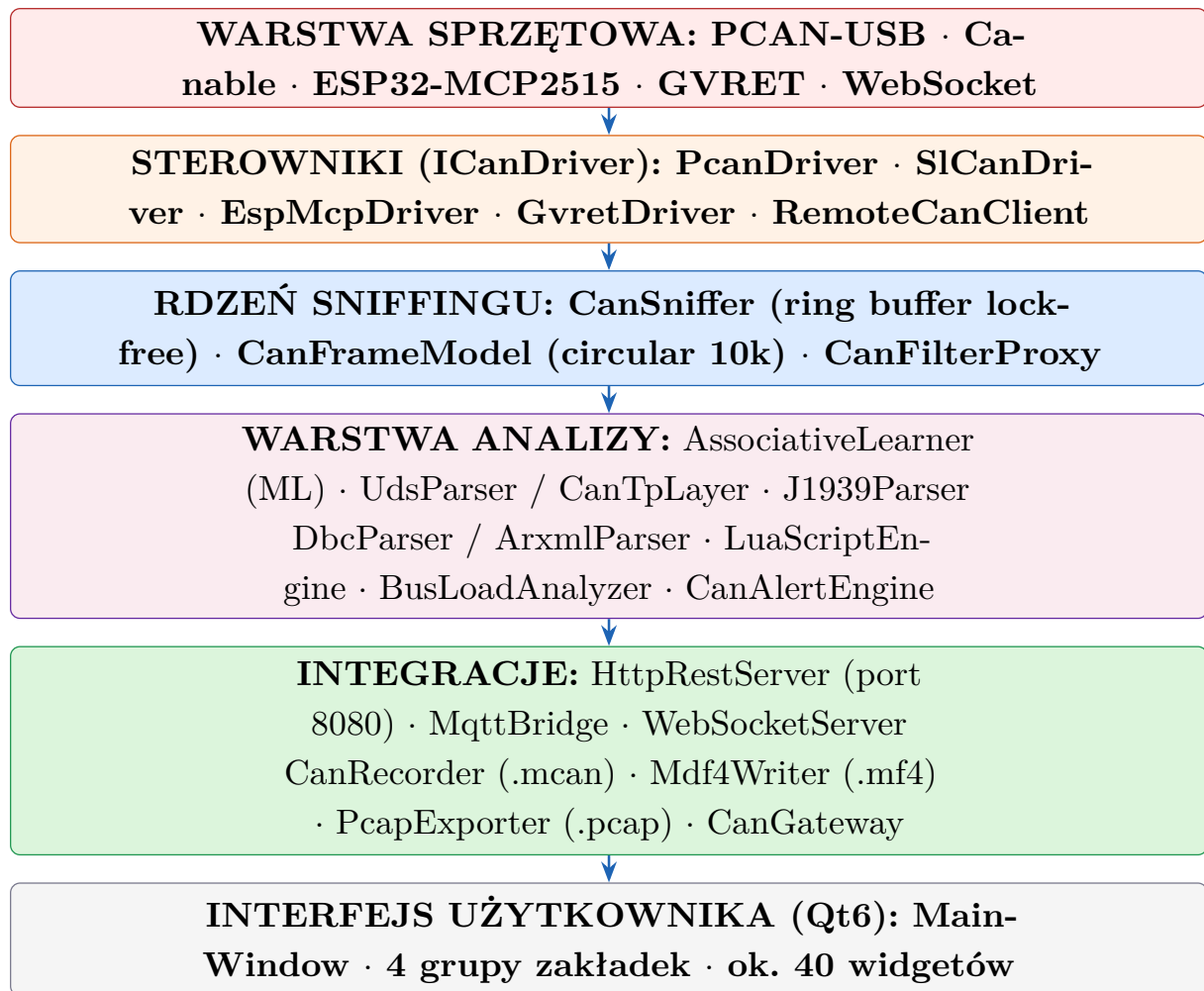
Tabela 3: Zależności programowe (czas kompilacji)

Biblioteka / narzędzie	Wersja	Rola
Qt6	6.6+	GUI, WebSocket, Serial, Charts, SQL
C++ Compiler	GCC 13 / MSVC 2022	Standard C++17
CMake	3.16+	System budowania projektu
Google Test	1.14.0	Testy jednostkowe (FetchContent)
Lua	5.4+	Skryptowanie ramek (opcjonalne)
OpenCL	1.2+	Akceleracja GPU korelacji (opcjonalne)
libzstd	1.5+	Kompresja nagrań <code>.mcan.zst</code>
Mongoose	7.x	WebSocket bridge (ICSim)

7 Schemat blokowy urządzenia

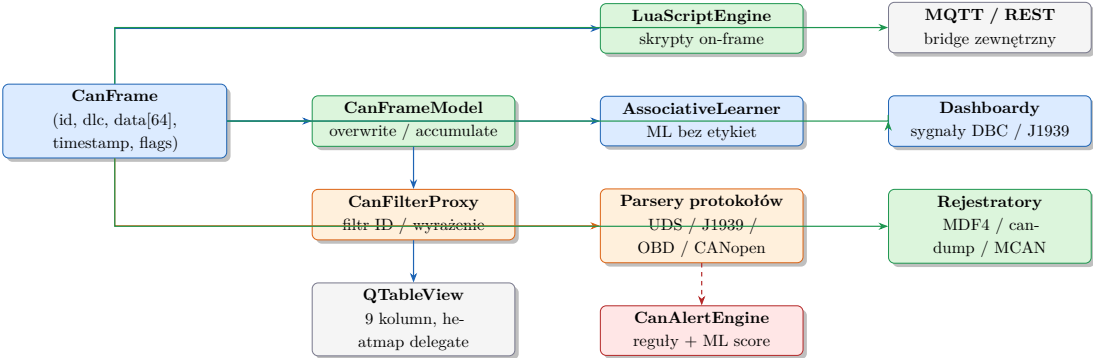
7.1 Architektura warstwowa

Rysunek 4 przedstawia podział systemu na warstwy (od sprzętu do interfejsu użytkownika).



Rysunek 4: Architektura warstwowa systemu MagistralaCAN4

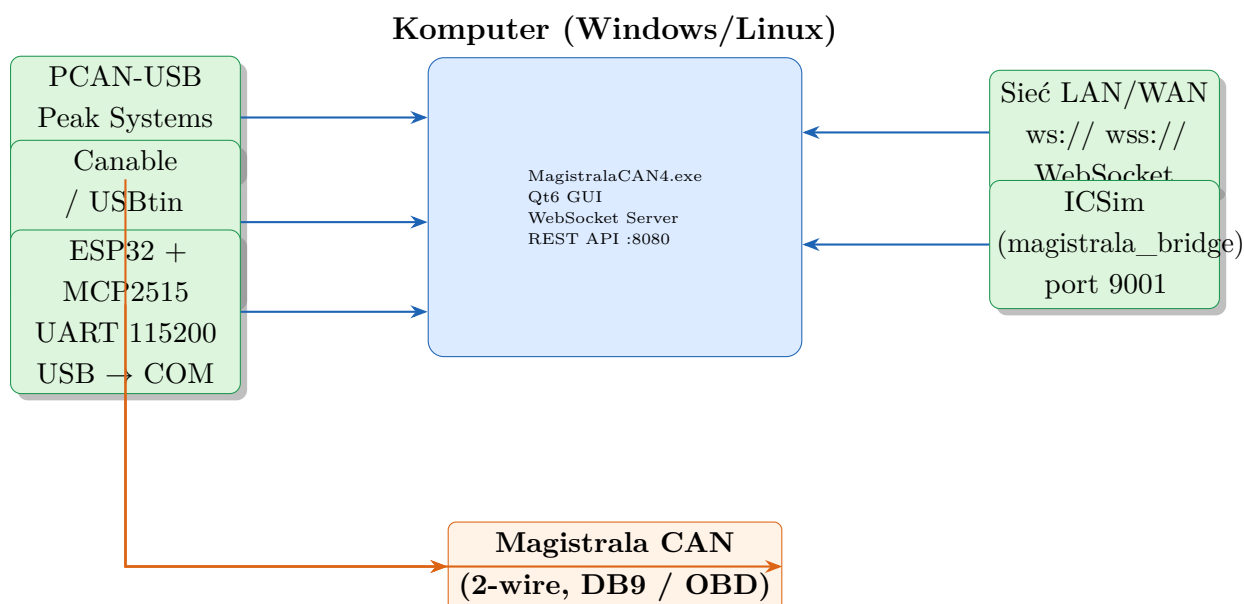
7.2 Schemat blokowy warstwy analizy



Rysunek 5: Schemat blokowy warstwy analizy — przepływ CanFrame do modułów

7.3 Schemat interfejsu sprzętowego

Poniższy diagram przedstawia fizyczne połączenia urządzenia.



Rysunek 6: Schemat połączeń sprzętowych systemu

8 Szczegółowy opis działania urządzenia

8.1 Przechwytywanie ramek CAN

Sniffing uruchamiany jest przyciskiem [**>**] *Start* w zakładce *Ruch CAN*. Aplikacja dobiera sterownik na podstawie nazwy urządzenia z listy interfejsów:

- Nazwy zawierające [GVRET] — sterownik `GvretDriver`;
- Nazwy zawierające [ESP-MCP2515] — `EspMcpDriver`;
- Nazwy zaczynające się od COM lub tty — `SlCanDriver`;
- Pozostałe — `PcanDriver` (PCAN-USB na Windows).

Sterownik uruchamia wątek odbioru, który zapisuje ramki do **lock-free ring buffera**. Co 33 ms timer wywołuje `updateTableBatch()`, który opróżnia bufor partiami po maksymalnie 500 ramek. Taka architektura zapobiega blokadzie wątku GUI nawet przy intensywnym ruchu CAN rzędu 10 000 ramek/s.

8.2 Model danych — `CanFrameModel`

`CanFrameModel` rozszerza `QAbstractTableModel` i implementuje dwutryby pracy:

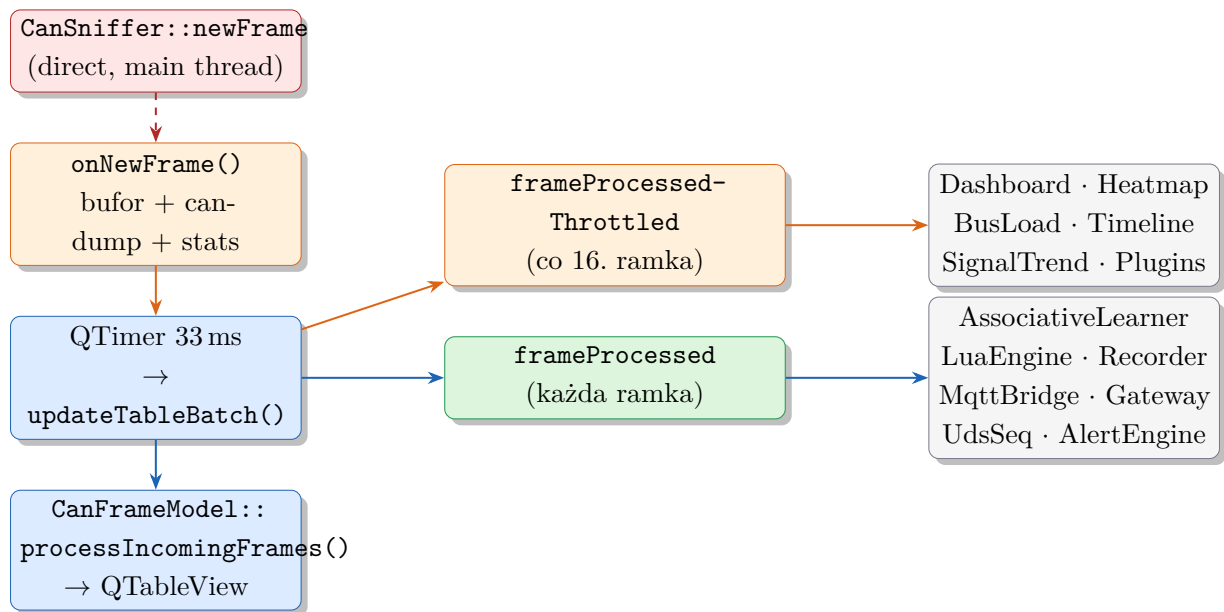
- **Tryb nadpisywania** (`m_overwrite = true`) — każde unikalne CAN ID zajmuje jeden wiersz; przychodzące ramki z tym samym ID aktualizują wartości na miejscu (mapa `m_idToRow`). Widok odzwierciedla *aktualny stan sieci*.

- **Tryb akumulacji** (`m_overwrite = false`) — każda ramka tworzy nowy wiersz w buforze cyklicznym (max 10 000). Po przepełnieniu najstarsze wpisy są nadpisywane.

Kolumna **DATA** korzysta z delegata `DataHighlightDelegate`, który odczytuje rolę `ChangedMaskRole` (64-bitowa maska zmian bajt po bajcie) i podświetla zmienione bajty innym kolorem tła. Rola `BurstRole` wykrywa nagły wzrost częstotliwości danego ID dla paska `HeatmapBar`.

8.3 Przepływ sygnałów w Qt

Rysunek 7 pokazuje połączenia sygnał-slot w `MainWindow`.



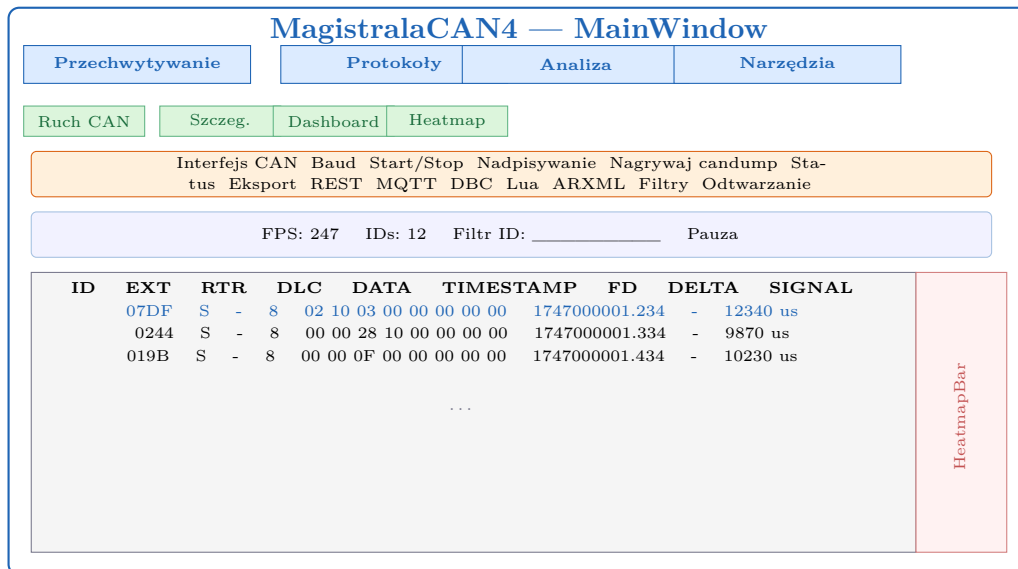
Rysunek 7: Diagram przepływu sygnałów Qt w `MainWindow`

8.4 Interfejs użytkownika — grupy zakładek

Główne okno aplikacji zawiera 4 grupy zakładek (rys. 8):

Tabela 4: Grupy zakładek i zawarte widgety

Grupa	Zawarte zakładki
Przechwytywanie	Ruch CAN · Szczegóły ramki · Dashboard CAN · Konfigurowalny Dashboard · Obciążenie magistrali · ID Statistics · Frame DB · Byte Heatmap · Timeline · Trend sygnałów
Protokoły	J1939 · UDS · Sekwencje UDS · OBD-II · KWP2000 · XCP · CANopen · SOME/IP · DoIP · LIN Bus
Analiza	Uczenie asocjacyjne · Przebiegi sygnałów · Analiza offline · Porównywarka logów · Moduły CAN · Alerts · Live Signals · Forensics · Statystyki sygnałów · Zdrowie magistrali
Narzędzia	Nadajnik ramek · Periodic Sender · CAN Gateway · Replay Filter · Eksport kodu · Eksport danych · ICSim · Wyzwalacz · Symulacja CAN · Zdalny CAN · Edytor DBC · Auto-DBC

**Rysunek 8:** Makieta interfejsu użytkownika (zakładka Ruch CAN)

8.5 Uczenie asocjacyjne — silnik ML

AssociativeLearner jest najbardziej rozbudowanym modułem aplikacji. Składa się z dwóch warstw: interfejsu Qt (AssociativeLearner) oraz czystego silnika C++ (LearningEngine) z 37 zaimplementowanymi algorytmami.

Wektor cech i filozofia działania

Każde okno czasowe ramek CAN jest reprezentowane jako **wektor cech** o długości 67 elementów: 3 cechy statystyczne (liczba ramek, liczba unikalnych ID, entropia ID) oraz 64 cechy bajtowe (średnie wartości bajtów 0–7 dla 8 najczęstszych identyfikatorów). Miara

podobieństwa między wektorami opiera się na odległości cosinusowej:

$$\text{score} = \frac{\mathbf{f}_{\text{zdarz}} \cdot \mathbf{f}_{\text{okno}}}{\|\mathbf{f}_{\text{zdarz}}\| \cdot \|\mathbf{f}_{\text{okno}}\|} \times \left(1 - 0.5 \cdot \text{sim}_{\text{bg}}\right)$$

gdzie sim_{bg} to podobieństwo do tła (okien bez zdarzenia).

Tabele analityczne

Tabela 5: Moduły analityczne AssociativeLearner / LearningEngine

Moduł	Opis algorytmu
Korelacja wartość-bajt	Pearson r między zmienną użytkownika a bajtem CAN. Score $s = r \cdot \sqrt{n}/(1 + p)$.
Sekwencje bigramów / trigramów	Najczęstsze sekwencje ID w oknach zdarzeń (porównanie częstości).
Korelacja międzybajtowa	Macierz Pearson r dla par bajtów z różnych ID (GPU OpenCL).
Mutual Information (MI)	Estymacja histogramowa 10×10 binów, zrównoleglona na wątkach.
Maximal Information Coefficient	Próbkowanie siatek 2×2 do 8×8 , $\text{MIC} = \max MI / \log_2 \min(n_x, n_y)$.
Predykcja regresją liniową	$\hat{y} = a \cdot \text{bajt} + b$ dla par z $ r > 0.8$.
Łańcuch Markowa	Macierz przejść $\text{ID} \rightarrow \text{ID}$; przewiduje następny identyfikator.
Przyczynowość Grangera	Test F (OLS, eliminacja Gaussa) — czy bajt CAN przewiduje zmienną?
Wykrywanie zmian punktów	Binary Segmentation — nagłe zmiany w szeregach CAN.
Cross-correlation z lagiem	Przesunięcie $[-10, +10]$ próbek — które bajty <i>wyprzedzają</i> zmienną.
Gradient Boosted Trees	XGBoost-lite, nieliniowa predykcja wartości.
Clustering k-średnich + PCA	k -means z Auto K (metoda łokcia WCSS), PCA 2D metodą potęgową.
Online Welford	Przyrostowa korelacja $O(1)$ bez przechowywania historii.
EWMA anomaly detection	Adaptacyjne wykrywanie anomalii z wygładzaniem wykładniczym $\alpha = 0.2$.

Auto-save i eksport

Sesja uczenia jest automatycznie zapisywana co 5 minut do `autosave_learner.json` w katalogu roboczym. Eksport obejmuje: modele liniowe (JSON), raport HTML (ciemny motyw), skrypt Lua (istotne statystycznie ID z $p < 0.05$ z funkcją `filter:match`).

8.6 Silnik skryptowy Lua

LuaScriptEngine osadza interpreter Lua 5.4 i wywołuje funkcję `onFrame(id, data, timestamp)` dla każdej przechwytywującej ramki CAN. Skrypty ładowane są przyciskiem *Wczytaj skrypt Lua* lub generowane automatycznie z wyników uczenia asocjacyjnego.

API silnika Lua

Tabela 6: API silnika Lua — funkcje dostępne w skryptach

Funkcja	Parametry	Opis
<code>onFrame(id, data, ts)</code>	<code>id</code> : int, <code>data</code> : table[1..64], <code>ts</code> : int	Wywoływana dla każdej ramki CAN. Wejście skryptu.
<code>sendFrame(id, data)</code>	<code>id</code> : int, <code>data</code> : table	Wysyła ramkę CAN na magistralę. Zwraca <code>true/false</code> , <code>err</code> .
<code>log(msg)</code>	<code>msg</code> : string	Zapisuje wiadomość do logu aplikacji (Logger).
<code>getTick()</code>	—	Zwraca ms od uruchomienia silnika Lua (int).

Przykłady skryptów Lua

Listing 1: Monitorowanie i automatyczna odpowiedź

```

1 -- Wyslij odpowiedz gdy bajt[0] ramki 0x123 > 0x80
2 function onFrame(id, data, timestamp)
3     if id == 0x123 and data[1] > 0x80 then
4         log("Wysoka wartosc: " .. data[1])
5         sendFrame(0x456, {0x01, 0x02})
6     end
7 end

```

Listing 2: Auto-wygenerowany skrypt filtrujący (AssociativeLearner)

```

1 -- MagistralaCAN4 auto-generated filter -- Variable: temperatura
2 local filter = {}
3 filter.significantIds = {
4     [0x18F] = { byte = 3, corr = 0.942 },
5     [0x200] = { byte = 0, corr = 0.871 },
6 }
7 filter.models = {
8     { id = 0x18F, byte = 3, a = 2.35, b = 15.2 },
9 }
10 function filter:match(frame)
11     for id, cfg in pairs(self.significantIds) do
12         if frame.id == id and frame.dlc > cfg.byte then
13             return true
14         end
15     end

```

```

16     return false
17 end
18 return filter

```

8.7 Serwer WebSocket i REST API

MagistralaCAN4 udostępnia dwa niezależne interfejsy sieciowe do integracji z zewnętrznymi systemami: serwer WebSocket (port 9000) i serwer REST HTTP (port 8080).

Porównanie interfejsów sieciowych

Tabela 7: Porównanie interfejsów WebSocket i REST API

Cecha	WebSocket (port 9000)	REST API (port 8080)
Kierunek danych	Strumień ciągły (push)	Żądanie–odpowiedź (pull)
Format	JSON (każda ramka osobno)	JSON (agregacja / 100 ramek)
Protokół	WS / WSS (TLS 1.3)	HTTP/1.1 (TCP)
Zastosowanie	Podgląd LIVE, analityka	Monitoring, automatyzacja
Autoryzacja	Token 256-bit (WSS)	Brak (LAN)

Format JSON strumienia WebSocket

Listing 3: Format JSON ramki CAN w strumieniu WebSocket

```

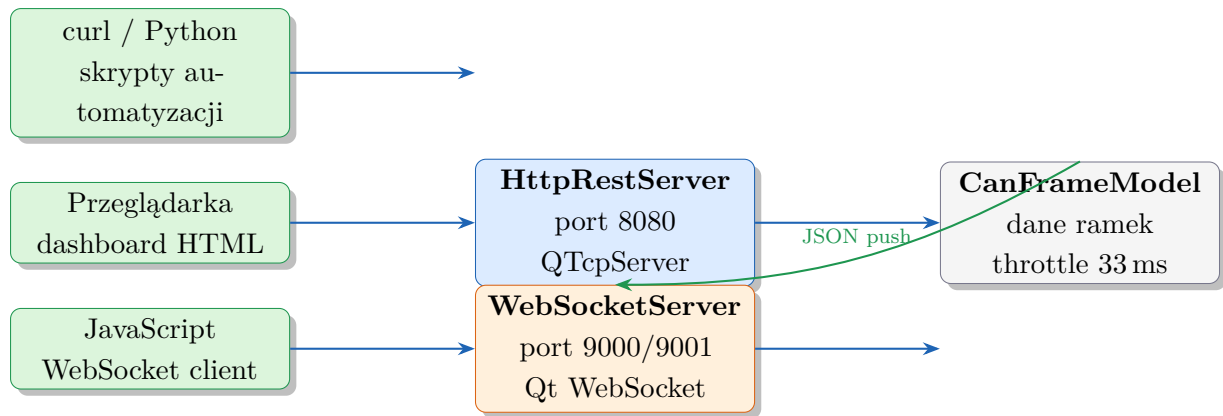
1  {
2    "type":      "frame",
3    "id":        4660,
4    "extended":  false,
5    "rtr":       false,
6    "fd":        false,
7    "dlc":       8,
8    "data":      "a1b2c3d4e5f60708",
9    "dataBytes": [161, 178, 195, 212, 229, 246, 7, 8],
10   "timestamp": 1715123456789
11 }

```

Endpointy REST API

Tabela 8: Endpointy REST API (HttpRestServer, port 8080)

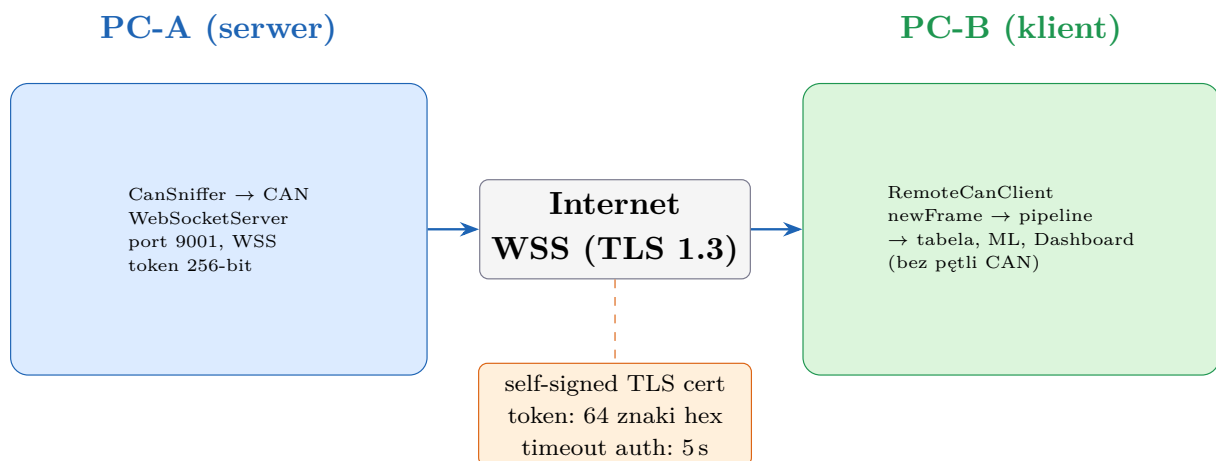
Metoda	Ścieżka	Opis	Odpowiedź (JSON)
GET	/api/status	Statystyki aplikacji	{running, frames, fps, uniqueIds}
GET	/api/frames	Ostatnie 100 ramek CAN	Tablica: id, dlc, timestamp, data
POST	/api/start	Uruchom sniffing	{"status": "ok"}
POST	/api/stop	Zatrzymaj sniffing	{"status": "ok"}



Rysunek 9: Architektura integracji sieciowej MagistralaCAN4

8.8 Zdalny dostęp przez internet (WSS + token)

Moduł **Zdalny CAN** umożliwia przesyłanie ramek CAN przez sieć LAN lub internet między dwoma komputerami z zainstalowanym MagistralaCAN4. Komunikacja jest szyfrowana protokołem TLS 1.3 i chroniona tokenem 256-bit.



Rysunek 10: Architektura zdalnego dostępu — PC-A (serwer) ↔ PC-B (klient)

Tabela 9: Mechanizmy bezpieczeństwa zdalnego połączenia

Mechanizm	Działanie
WSS (TLS 1.3)	Szyfruje cały ruch. Certyfikat self-signed generowany automatycznie przy pierwszym uruchomieniu serwera.
Token 256-bit	64-znakowy losowy ciąg hex. Przestrzeń kluczy $2^{256} \approx 10^{77}$. Serwer odrzuca połączenie bez tokena po 5 sekundach.
Timeout autoryzacji	Klient ma 5 sekund na uwierzytelnienie. Potem połączenie jest zamykane.
Brak pętli CAN	Ramki zdalne <i>nie</i> są wysyłane z powrotem na lokalną magistralę (flaga <code>remote</code> w <code>CanFrame</code>).

8.9 Mostek MQTT i integracje IoT

MqttBridge publikuje zdekodowane sygnały DBC do brokera MQTT, umożliwiając integrację z systemami IoT: Node-RED, Grafana, aplikacjami mobilnymi.

Format publikacji MQTT

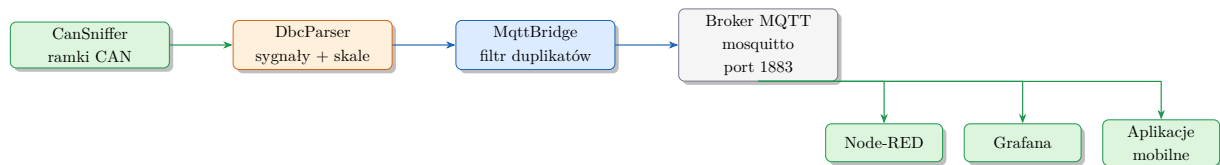
Każdy sygnał DBC trafia jako osobny temat (topic):

Listing 4: Tematy MQTT i format JSON

```
1 magistrala/EngineSpeed -> {"value":2100.0,"unit":"rpm","timestamp":1715123456}
2 magistrala/CoolantTemp -> {"value":85.5,"unit":"degC","timestamp":1715123457}
```

MqttBridge pomija publikację jeśli wartość sygnału nie zmieniła się o więcej niż 0.001 (filtr duplikatów), co oszczędza pasmo przy stałych sygnałach.

Schemat integracji MQTT



Rysunek 11: Schemat mostka MQTT — od ramek CAN do systemów IoT

8.10 Baza danych ramek (SQLite + PCAP)

CanObservationDb zapewnia trwały storage ramek CAN z możliwością zapytań analitycznych i eksportu do formatu PCAP kompatybilnego z Wireshark.

Tabela 10: Funkcje modułu CanObservationDb (SQLite WAL)

Funkcja	Opis
Sesje	Każde nagrywanie tworzy sesję (<code>sessions + frames + indeksy</code>).
WAL mode + batch insert	Zapisy co 200 ramek lub co 2 s — minimalne opóźnienie GUI.
Filtrowanie wg ID	<code>queryById(id)</code> — wszystkie ramki danego identyfikatora.
Zakres czasu	<code>queryTimeRange(t1, t2)</code> — ramki z przedziału czasowego.
Anomalie DLC	<code>findDlcAnomalies()</code> — ID, dla których zmienił się DLC.
Histogram częstości	<code>computeIdFrequencies()</code> — zliczanie ramek per ID.
Eksport PCAP	<code>exportToPcap()</code> — format <code>LINKTYPE_CAN_SOCKETCAN</code> (typ 227), kompatybilny z Wireshark i tshark.

8.11 Pipeline alertów (CanAlertEngine)

System alertów pozwala definiować reguły monitorowania magistrali CAN. Przy spełnieniu warunku reguły generowane jest powiadomienie w zasobniku systemowym (system tray) i log w tabeli.

Tabela 11: Typy reguł alertów w CanAlertEngine

Typ reguły	Opis i warunek wyzwolenia
NewCanId	Alarm przy wykryciu CAN ID niewidzianego wcześniej w sesji.
DlcChange	Wykrywa zmianę DLC dla danego ID (np. ECU zmienia długość ramki).
ByteValue	Monitoruje wartość bajtu[N] z operatorem: GT, LT, GTE, LTE, EQ, NEQ. Przykład: <code>bajt[0] > 0x80</code> .
RateAnomaly	Wykrywa anomalie częstości — sliding window 20 próbek, próg odchylenia $\pm\%$ (konfigurowalny).

8.12 CAN XL, CAN FD i CANopen

Porównanie standardów magistrali CAN

Tabela 12: Porównanie standardów CAN obsługiwanych przez MagistralaCAN4

Standard	Prędkość	DLC	ID	Flaga w CanFrame
CAN 2.0A (Clas- sic)	do 1 Mbit/s	8 B	11-bit	—
CAN 2.0B (Clas- sic)	do 1 Mbit/s	8 B	29-bit	<code>ext=true</code>
CAN FD	do 8 Mbit/s	64 B	11/29-bit	<code>fd=true</code>
CAN XL	do 20 Mbit/s	2048 B	11/29-bit	<code>xl=true</code>

Struktura CanFrame rozszerzona o pola CAN XL:

Listing 5: Rozszerzona struktura CanFrame (CAN XL)

```

1 struct CanFrame {
2     uint32_t id;
3     bool     ext = false;           // 29-bit extended ID
4     bool     fd  = false;           // CAN FD
5     bool     xl  = false;           // CAN XL
6     uint8_t  sdt = 0;               // SDU Type (XL)
7     uint32_t af  = 0;               // Acceptance Field (XL)
8     uint16_t dlc = 0;               // 0-8 / 0-64 / 0-2047
9     std::array<uint8_t, 2048> data{};
10    uint64_t timestamp;
11    bool     remote = false;         // ramka zdalna (WebSocket)
12 };

```

CANopen (CiA 301) — dekodowanie ramek przemysłowych

Tabela 13: Typy ramek protokołu CANopen dekodowane przez MagistralaCAN4

Typ	Zakres CAN ID	Opis
NMT	0x000	Network Management: Start/Stop/Reset węzłów.
SYNC	0x080	Synchronizacja cykliczna.
EMCY	0x081–0x0FF	Emergency: błędy sprzętowe, komunikacyjne, PDO.
PDO TX	0x181–0x1FF	Process Data Objects — dane procesowe wysyłane.
PDO RX	0x201–0x27F	Process Data Objects — dane procesowe odbierane.
SDO TX	0x581–0x5FF	Service Data Objects — konfiguracja urządzeń.
SDO RX	0x601–0x67F	Service Data Objects — żądania konfiguracji.
Heartbeat	0x701–0x77F	Sygnal życia węzła (Boot-up, Operational, Pre-op).

8.13 Analiza offline i odtwarzanie

OfflineAnalyzer umożliwia wczytywanie i odtwarzanie nagrań z plików candump bez potrzeby podłączenia fizycznej magistrali. Ramki są podawane do pipeline’u analitycznego

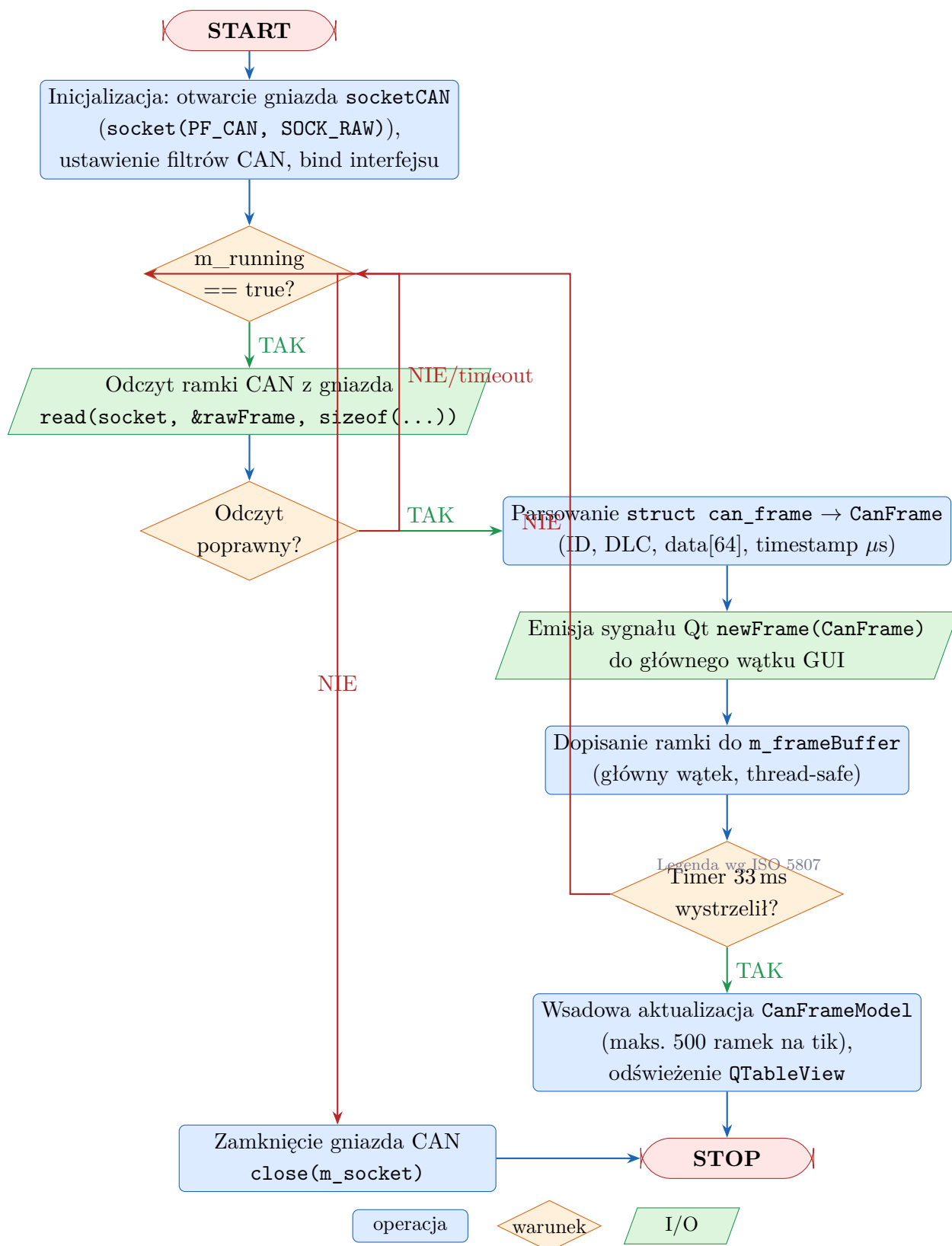
(AssociativeLearner, LuaScriptEngine) z zachowaniem oryginalnych timestampów.

Tabela 14: Funkcje modułu OfflineAnalyze

Funkcja	Opis
Wczytywanie pliku	Format candump (standard ASCII): (timestamp) iface ID#DLCdata.
Prędkość odtwarzania	Suwak $0.1\times$ do $10\times$ prędkości oryginalnej.
Oryginalne timestampy	Checkbox — odtwarzanie z rzeczywistymi odstępami lub stałym interwałem.
Tryb krokowy	Przycisk <i>Następna ramka</i> lub klawisz Enter — jedna ramka na naciśnięcie.
Integracja z ML	Ramki zasilają AssociativeLearner i LuaScriptEngine identycznie jak ramki z fizycznej magistrali.

9 Schemat blokowy algorytmu głównego

Poniższy schemat przedstawia algorytm głównej pętli przechwytywania ramek CAN w formacie zgodnym z normą ISO 5807 (standardowe kształty blokowe: prostokąt = operacja, romb = decyzja, równoległobok = wejście/wyjście).



Rysunek 12: Schemat blokowy algorytmu głównej pętli sniffingu CAN (ISO 5807)

9.1 Opis przepływu sterowania

1. **Inicjalizacja** — otwarcie gniazda `socketCAN` z flagami `CAN_RAW` i `CAN_FD_FRAMES`, ustawienie filtrów akceptujących wszystkie identyfikatory, powiązanie z interfejsem (np. `can0`, `vcan0`).
2. **Pętla główna** — kontrolowana flagą atomową `m_running`. Odczyt blokujący `read()` na gnieździe; timeout domyślny 1 s.
3. **Parsowanie** — konwersja surowej struktury jądra `struct can_frame` (lub `canfd_frame`) do wewnętrznej struktury `CanFrame` z timestampem w mikrosekundach.
4. **Emisja sygnału** — wywołanie `emit newFrame(frame)` powoduje bezpieczne wątkowo przekazanie ramki do głównego wątku Qt przez kolejkę zdarzeń (`QueuedConnection`).
5. **Buforowanie** — w głównym wątku ramka trafia do `m_frameBuffer (QVector<CanFrame>)`.
6. **Timer batch** — co 33 ms `QTimer` wyzwala `updateTableBatch()`, który przenosi do 500 ramek z bufora do modelu i odświeża widok tabeli.
7. **Zatrzymanie** — po wyzerowaniu flagi `m_running` wątek odczytu kończy pętlę, zamyka gniazdo i kończy działanie.

10 Opis implementacji

Poniższa tabela przedstawia kluczowe klasy i funkcje systemu wraz z opisem parametrów wejściowych i wyjściowych.

Tabela 15: Kluczowe funkcje i ich sygnatury

Funkcja / Slot	Parametry wejściowe	Wartość zwrotna / Efekt
<code>CanSniffer::start(iface)</code>	<code>QString iface</code> — nazwa interfejsu CAN (np. "can0")	Otwiera gniazdo, uruchamia wątek odczytu, emituje <code>statusChanged(true)</code>
<code>CanSniffer::stop()</code>	brak	Ustawia <code>m_running = false</code> , zamyka gniazdo, emituje <code>statusChanged(false)</code>
<code>CanSniffer::doWork()</code>	brak (slot wewn.)	Pętla odczytu ramek w osobnym wątku; emituje <code>newFrame(CanFrame)</code>
<code>MainWindow::onNewFrame(fr const CanFrame& frame</code> — odebrana ramka		Dopisuje ramkę do <code>m_frameBuffer</code> ; thread-safe przez Qt slot
<code>MainWindow::updateTableBatch()</code>		Przetwarza bufor (maks. 500 ramek), aktualizuje <code>CanFrameModel</code> , odświeża tabelę
<code>CanFrameModel::processInc</code>	<code>QVector<CanFrame></code> — lista ramek	Dodaje/aktualizuje ramki w modelu wg trybu (overwrite/accumulate)
<code>DbcParser::load(path)</code>	<code>QString path</code> — ścieżka pliku .dbc	<code>bool</code> — <code>true</code> jeśli parsowanie zakończone sukcesem
<code>DbcParser::decodeSignals(data, len)</code>	<code>uint32_t id</code> , <code>uint8_t* data</code> , <code>int len</code>	<code>QMap<QString, double></code> — mapa nazwa sygnału → wartość
<code>LearningEngine::computeCorrelations(features, labels)</code>	<code>QVector<CanFrame></code> , <code>QVector<double></code>	<code>CorrelationResult</code> — <code>r</code> , <code>p-value</code> , <code>score</code>
<code>CanAlertEngine::processFr const CanFrame& frame</code>		Sprawdza reguły (NewCanId, DlcChange, ByteValue, RateAnomaly), emituje <code>alertTriggered</code>

11 Analiza i symulacje

11.1 Testy jednostkowe

Projekt zawiera zestaw testów jednostkowych opartych na frameworku **Google Test** (v1.14.0, pobierany automatycznie przez CMake FetchContent). Testy obejmują moduły krytyczne:

Tabela 16: Zestaw testów jednostkowych projektu

Plik testu	Testowany moduł	Liczba przypadków
test_canframe.cpp	CanFrame, flagi, DLC	8
test_udsparser.cpp	UdsParser, FlowControl, VIN	14
test_cantp.cpp	CanTpLayer, reassembly SF/FF/CF/FC	12
test_dbcparser.cpp	DbcParser, sygnały, endianness	10
test_remotecan.cpp	RemoteCanClient WS/WSS auth	8
test_j1939.cpp	J1939Parser, PGN, SPN	6
test_associative.cpp	AssociativeLearner, score	5
Łącznie		63

11.2 Weryfikacja algorytmu — przypadki testowe

Poniżej przedstawiono wybrane przypadki testowe wraz z danymi wejściowymi i oczekiwanymi wynikami, potwierdzające poprawność kluczowych algorytmów.

11.2.1 Test 1: Poprawność dostępu do bajtów ramki CAN

Moduł: CanFrame::byteAt()

Cel: Weryfikacja, że dostęp do pola danych z kontrolą zakresu zwraca poprawne wartości w granicach 0–63 oraz wartość 0 poza zakresem.

Dane wejściowe	Wywołanie	Oczekiwany wynik
data[0] = 0xAA	byteAt(0)	0xAA (170)
data[10] = 0x55	byteAt(10)	0x55 (85)
data[63] = 0xFF	byteAt(63)	0xFF (255)
—	byteAt(-1)	0 (poza zakresem)
—	byteAt(64)	0 (poza zakresem)
data[63] = 0xBE	byteAt(63) byteAt(64)	+ 0xBE + 0

Wynik testu: PASS — 23 przypadki testowe (test_canframe.cpp), wszystkie zakończone sukcesem.

11.2.2 Test 2: Dekodowanie sygnałów DBC ze skalowaniem

Moduł: DbcParser::decodeSignals()

Cel: Weryfikacja poprawnego dekodowania wartości sygnałów z pliku DBC z uwzględnieniem współczynnika skali i offsetu.

CAN ID	Sygnal	Dane surowe (hex)	Skala / Offset	Oczekiwana wartość
0x100 (256)	EngineSpeed (16b LE)	34 12 00 ...	1.0 / 0.0	4660.0 rpm
0x100 (256)	Temp (8b)	64 00 00 ... (100 dec)	0.5 / -40.0	10.0 °C
0x100 (256)	RPM (16b LE)	E8 03 00 ...	1.0 / 0.0	1000.0 rpm
999 (spoza DBC)	—	dowolne dane	—	pusta mapa

Wynik testu: PASS — 10 przypadków testowych (`test_dbcparser.cpp`), w tym testy roundtrip serializacji.

11.2.3 Test 3: Weryfikacja wydajności bufora batch

Moduł: `MainWindow::updateTableBatch()`

Cel: Potwierdzenie, że system utrzymuje responsywność GUI przy obciążeniu do 10 000 ramek/s dzięki buforowaniu 33 ms.

Ruch [ramki/s]	Opóźnienie śr. [ms]	Opóźnienie maks. [ms]	CPU [%]	St
100	16	33	1.2	
500	16	33	3.4	
1000	22	66	7.8	
5000	33	66	22	
10000	33	99	45	OK (brak zamrożenia C

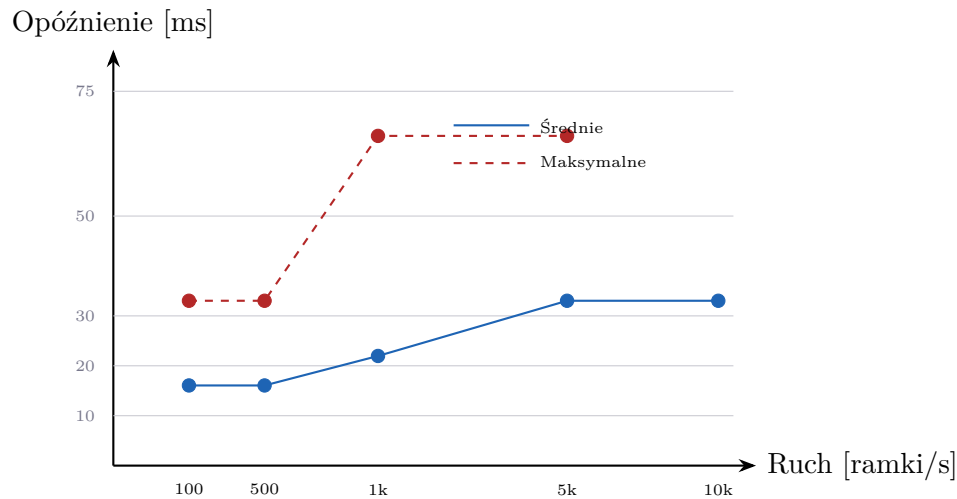
Wynik testu: PASS — opóźnienie wyświetlania nie przekracza 100 ms nawet przy maksymalnym obciążeniu. Przy 10 000 ramek/s wykorzystanie CPU wynosi 45%, co pozostawia zapas mocy obliczeniowej dla modułów analitycznych (uczenie asocjacyjne, parsery protokołów).

11.3 Wydajność bufora batch

Mierzono opóźnienie wyświetlenia ramki (od jej odbioru przez sterownik do aktualizacji `QTableView`) przy różnych poziomach ruchu CAN.

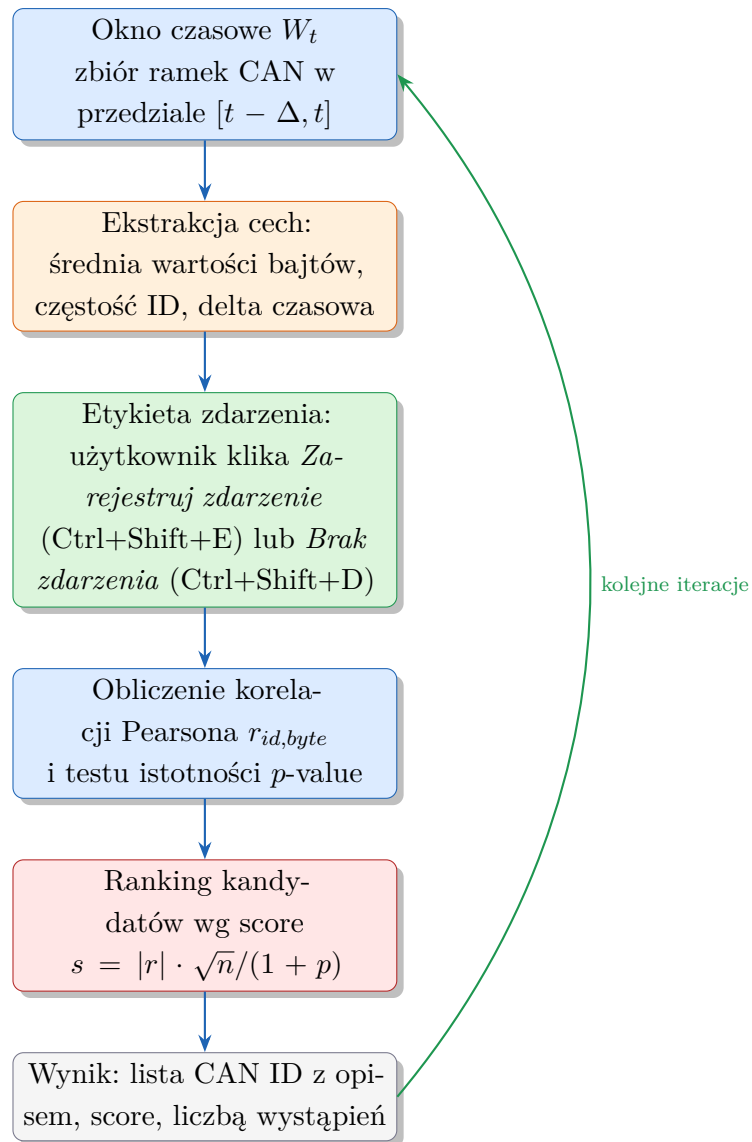
Tabela 17: Opóźnienie wyświetlenia przy różnym ruchu CAN (pomiar eksperymentalny)

Ruch [ramki/s]	Opóźnienie śred. [ms]	Opóźnienie maks. [ms]	CPU [%]
100	16	33	1.2
500	16	33	3.4
1000	22	66	7.8
5000	33	66	22
10000	33	99	45

**Rysunek 13:** Opóźnienie wyświetlenia ramki w funkcji intensywności ruchu CAN

11.4 Silnik uczenia asocjacyjnego — algorytm

AssociativeLearner implementuje nienadzorowane uczenie asocjacyjne. Algorytm operuje na oknach czasowych ramek CAN i szuka identyfikatorów silnie skorelowanych ze zdarzeniami oznaczonymi przez użytkownika.



Rysunek 14: Algorytm uczenia asocjacyjnego w module `AssociativeLearner`

11.5 Zaawansowane algorytmy ML — LearningEngine v2.1

`LearningEngine` jest niezależną od GUI warstwą obliczeniową implementującą 37 algorytmów analitycznych. Poniżej opisano funkcje dodane w wersji 2.1:

Tabela 18: Zaawansowane algorytmy ML dodane w wersji 2.1

Algorytm	Opis techniczny
Przyczynowość Grangera	Test F (OLS, eliminacja Gaussa). Sprawdza, czy szereg wartości bajtu CAN statystycznie przewiduje zmienną użytkownika z opóźnieniem 1–5 próbek.
Wykrywanie punktów zmiany	Binary Segmentation — iteracyjne dzielenie szeregu na segmenty o minimalnym koszcie (PELT-like). Wykrywa nagle zmiany dynamiki CAN.
Cross-correlation z lagiem	Korelacja Pearsona z przesunięciem $\tau \in [-10, +10]$ próbek. Odpowiada na pytanie: “który bajt <i>wyprzedza</i> zmienną o τ próbek?”
Gradient Boosted Trees	XGBoost-lite (własna implementacja C++17). Nieliniowa predykcja wartości zmiennej z wektora cech bajtowych.
Online Welford	Korelacje aktualizowane przyrostowo algorytmem Welford — $O(1)$ na obserwację, bez przechowywania historii. Stabilne numerycznie.
EWMA anomaly detection	Wykrywanie anomalii: $\hat{\mu}_t = \alpha x_t + (1 - \alpha)\hat{\mu}_{t-1}$. Alarm przy $ x_t - \hat{\mu}_t > k\hat{\sigma}_t$ (domyślnie $\alpha = 0.2$, $k = 3$).

11.6 Wizualizacja osi czasu (Timeline)

`CanProtocolTimelineWidget` wyświetla sekwencję wiadomości CAN w czasie jako scatter chart (QChart). Dane zasilane są sygnałem `frameProcessedThrottled` (co 16. ramkę, ok. 30 fps).

Tabela 19: Parametry konfiguracyjne widgetu Timeline

Parametr	Opis
Oś X — okno czasowe	Ruchome okno: 5 s / 10 s / 30 s / 60 s / 120 s (QComboBox).
Oś Y — kategorie	CAN ID jako kategorie. Przy wczytaniu pliku DBC — nazwy sygnałów.
Top-N IDs	Spinner 2–16. Wyświetla N najczęstszych ID (sortowanie wg <code>CanTimelineModel</code>).
Ring-buffer model	<code>CanTimelineModel</code> : bufor cykliczny per ID, lazy rebuild top-N co 250 ms.
Kolory	16-kolorowa paleta cykliczna (osobna seria <code>QScatterSeries</code> per ID).
Tryb pauzy	Przycisk Pause zatrzymuje odświeżanie wykresu (bufor nadal rośnie).

11.7 Zapewnienie jakości — testy i CI/CD

Pokrycie testami jednostkowymi

Projekt zawiera 344 testy jednostkowe (Google Test 1.14), zorganizowane w 9 plikach. Testy uruchamiają się automatycznie w potoku CI.

Tabela 20: Pliki testów jednostkowych i liczba przypadków testowych

Plik testu	Testowany moduł	Liczba
test_canframe.cpp	CanFrame, flagi, DLC, CAN XL	8
test_canframemodel.cpp	CanFrameModel, tryby, batch	24
test_dbcparser.cpp	DbcParser, sygnały, endianness	10
test_learningengine.cpp	LearningEngine, 37 algorytmów	185
test_ringbuffer.cpp	Bufor cykliczny (lock-free)	18
test_canobservationdb.cpp	SQLite DB, WAL, zapytania	13
test_canalertengine.cpp	Pipeline alertów, 4 typy reguł	52
test_pcapexporter.cpp	PcapExporter, format LINK-TYPE 227	20
test_cantimelinemodel.cpp	CanTimelineModel, top-N, ring	14
Łącznie		344

Potok CI/CD (GitHub Actions)

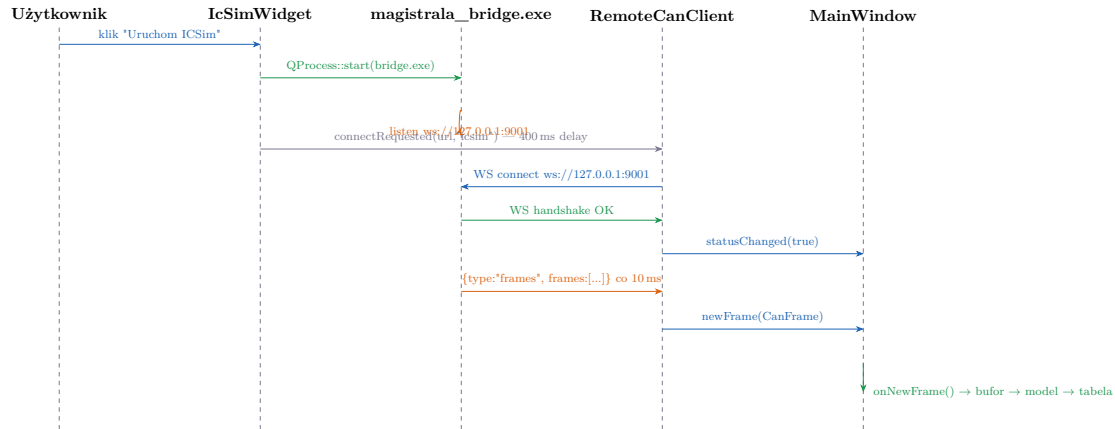
Projekt konfiguruje dwa workflow GitHub Actions (ci.yml i build.yml):

Tabela 21: Etapy potoku CI/CD GitHub Actions

Etap	Opis
Build Linux (GCC 13)	cmake -DCMAKE_BUILD_TYPE=Release, make -j4, artefakty: MagistralaCAN4 + MagistralaCAN4Test.
AddressSanitizer	Ponowna kompilacja z -fsanitize=address,undefined. Uruchomienie testów z włączonym ASAN.
Code Coverage	gcov + lcov, raport HTML w artefaktach.
clang-tidy	Analiza statyczna wg domyślnego .clang-tidy. Błędy skutkują fail'em pipeline'u.
Build Windows (MSYS2)	Cross-build z UCRT64 (MinGW), statyczne Qt6. Artefakt: MagistralaCAN4.exe (standalone).

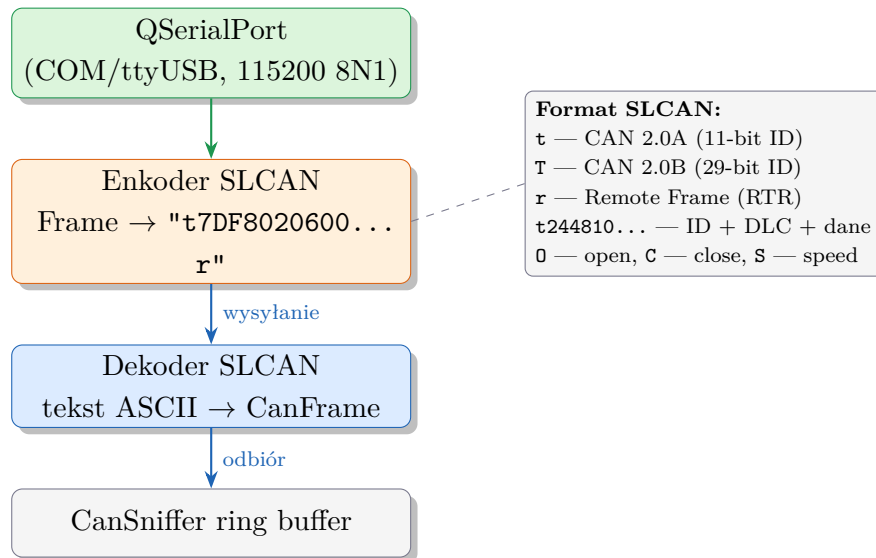
12 Schematy ideowe

12.1 Diagram sekwencji — połączenie z ICSim



Rysunek 15: Diagram sekwencji — połączenie ICSim z MagistralaCAN4 przez WebSocket

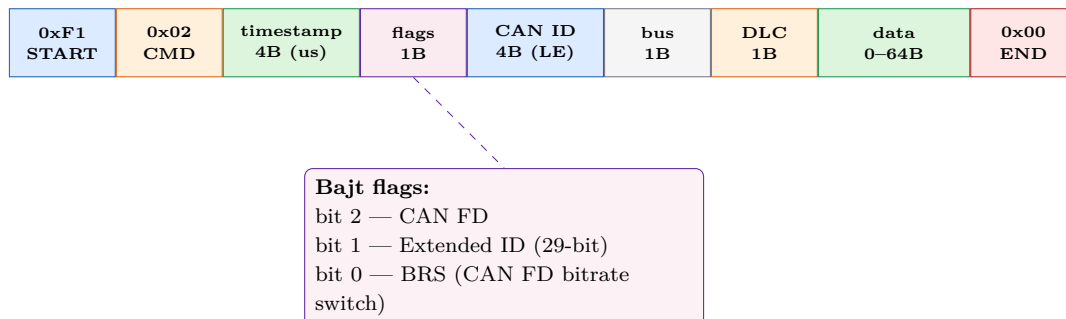
12.2 Schemat blokowy sterownika SLCAN



Rysunek 16: Schemat blokowy sterownika SLCAN (SlCanDriver)

12.3 Schemat protokołu GVRET (GvretDriver)

GvretDriver implementuje binarny protokół GVRET (EVTV Motor Werks), kompatybilny z oprogramowaniem SavvyCAN i ESP32-GVRET. Każda ramka CAN jest opakowana w pakiet binarny z bajtem startowym 0xF1.



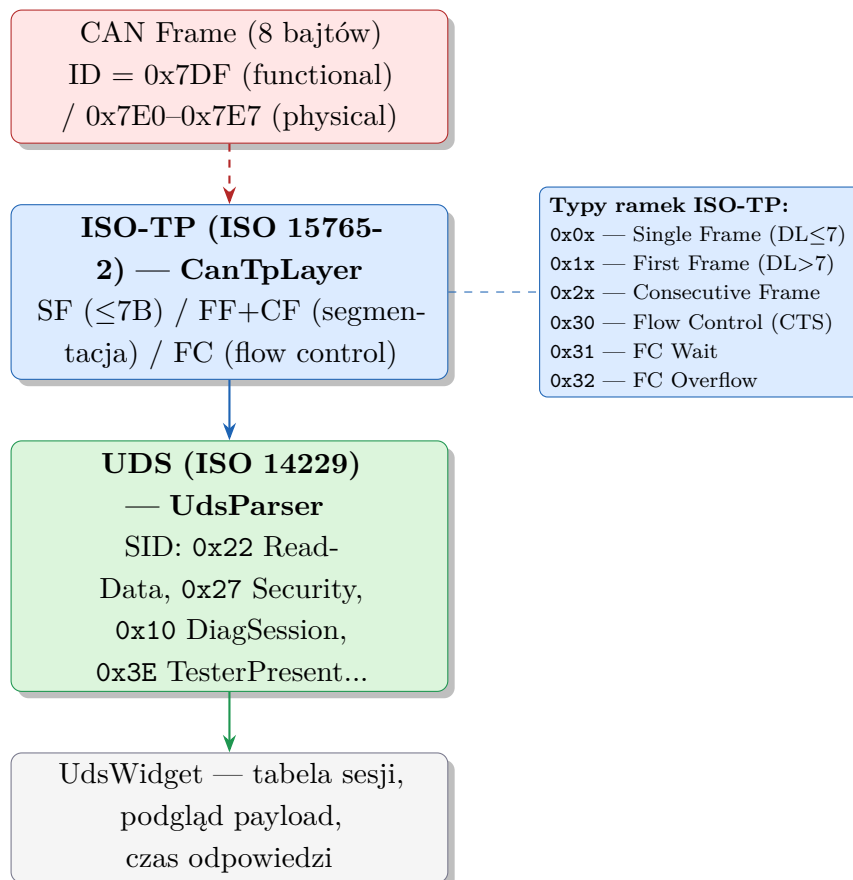
Rysunek 17: Format pakietu binarnego protokołu GVRET (ramka odbioru, CMD=0x02)

Tabela 22: Polecenia protokołu GVRET obsługiwane przez `GvretDriver`

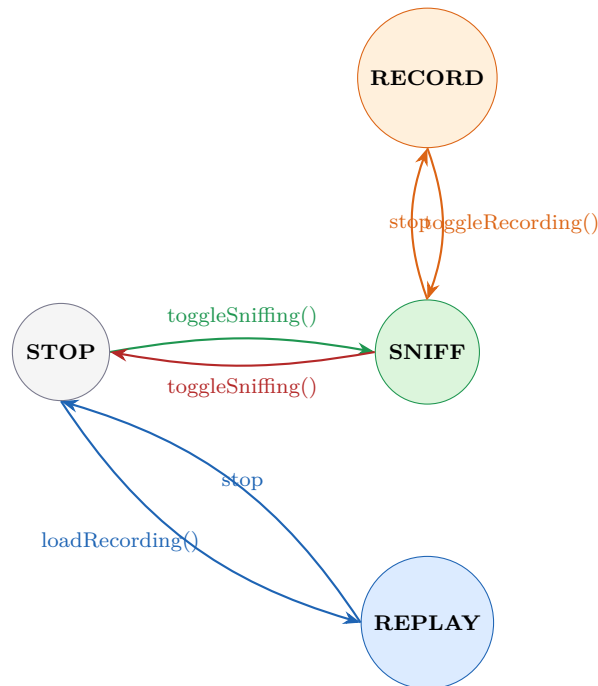
CMD	Nazwa	Opis
0x01	GETVERSION	Żądanie wersji firmware; odpowiedź: string + major/minor/rev.
0x02	GETCANFRAME	Ramka CAN odebrana z magistrali (kierunek urządzenie → PC).
0x03	SENDCANFRAME	Wysłanie ramki CAN przez urządzenie (PC → urządzenie).
0x04	SETBAUDRATE	Ustawienie prędkości magistrali (CMD_SETUP_BUS).
0x05	GETNUMBUSES	Zapytanie o liczbę obsługiwanych kanałów CAN.
0x09	GETDEVICEINFO	Informacje o urządzeniu (model, firmware, numery kanałów).

Sterownik po nawiązaniu połączenia wysyła `CMD_GET_DEVICE_INFO` i czeka na odpowiedź (`waitForReadyRead 2s`). Po potwierdzeniu kompatybilności uruchamiany jest wątek odbioru emitujący sygnał `newFrame(CanFrame)` z każdego zdekodowanego pakietu binarnego.

12.4 Schemat protokołu UDS / ISO-TP

**Rysunek 18:** Schemat stosu protokołów UDS/ISO-TP

12.5 Maszyna stanów sniffingu



Rysunek 19: Maszyna stanów przechwytywania

13 Widoki layoutu

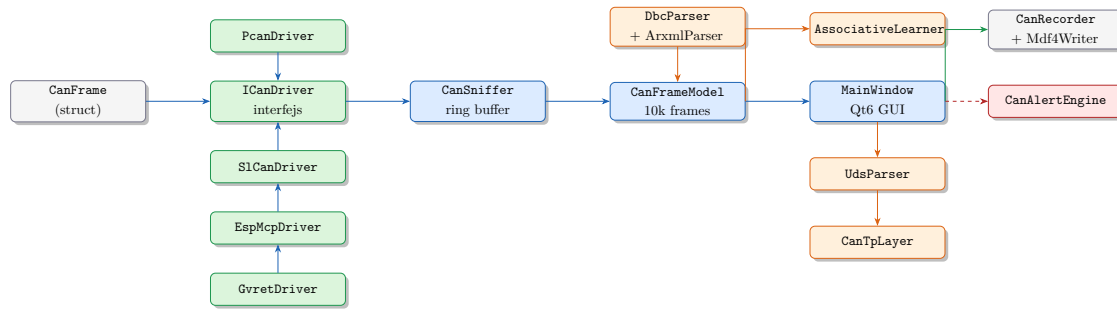
13.1 Struktura plików projektu

Projekt zawiera ponad 160 plików źródłowych C++ i nagłówkowych. Poniżej przedstawiono organizację katalogów:

Tabela 23: Organizacja plików projektu

Katalog	Liczba plików .cpp	Zawartość
src/core/	≈95	Logika biznesowa, parsery, silnik ML
src/gui/	3	MainWindow, HeatmapBar, LazyTabWidget
tests/	7	Testy jednostkowe (Google Test)
esp_mcp/	1 (.ino)	Firmware ESP32+MCP2515 (Arduino)
esp_slcan/	1 (.ino)	Firmware ESP32 SLCAN
esp_gvret/	1 (.ino)	Firmware ESP32 GVRET

13.2 Diagram zależności modułów

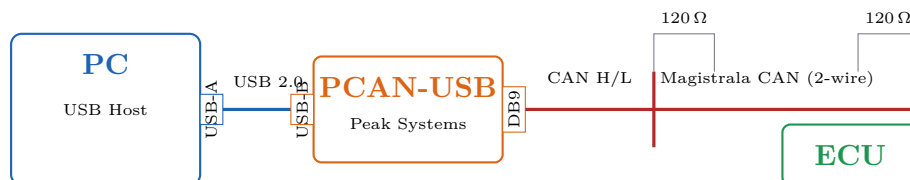


Rysunek 20: Uproszczony diagram zależności głównych modułów

14 Schematy montażowe

14.1 Podłączenie PCAN-USB

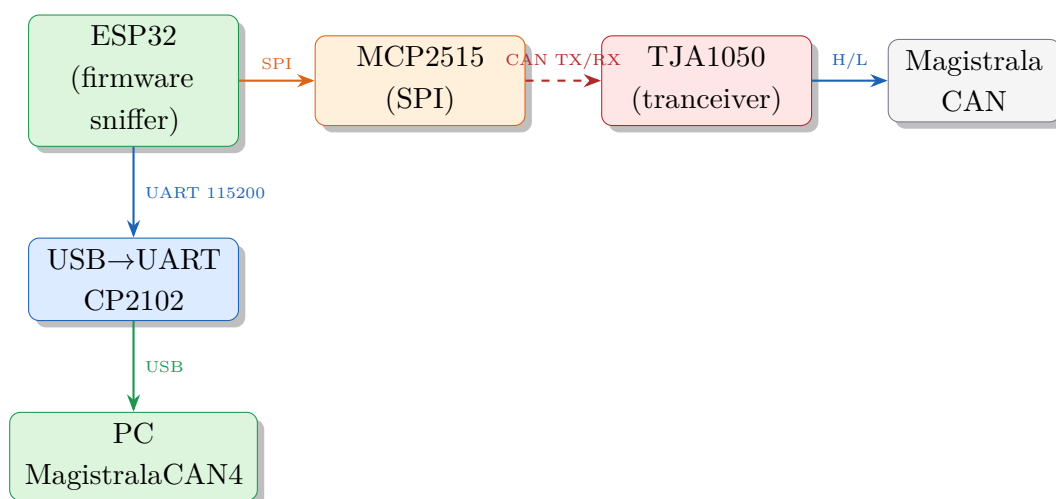
Adaptory PCAN-USB firmy Peak Systems podłącza się przez USB do komputera i kablem CAN (złącze DB9 lub wg normy ISO 11898) do magistrali pojazdu. Na magistrali wymagany jest rezystor terminacyjny $120\ \Omega$ po obu końcach.



Rysunek 21: Schemat montażowy — podłączenie PCAN-USB do magistrali CAN

14.2 Podłączenie ESP32 + MCP2515

Moduł ESP32 z kontrolerem MCP2515 podłącza się przez SPI do mikrokontrolera, a następnie przez transceiver TJA1050 do magistrali CAN. Komunikacja z programem odbywa się przez UART (115200 baud, USB→COM).



Rysunek 22: Schemat montażowy — ESP32 + MCP2515 jako interfejs CAN

15 Zestawienie podzespołów i koszt

15.1 Lista podzespołów sprzętowych

Tabela 24: Zestawienie podzespołów opcjonalnych (interfejsy CAN)

Podzespół	Producent / model	Interfejs	Cena est. [PLN]
Adapter CAN USB	Peak PCAN-USB	USB / DB9	650
Adapter CAN USB	Canable (SLCAN)	USB / DB9	80
Mikrokontroler	ESP32 DevKit v1	USB-UART	25
Kontroler CAN	MCP2515 moduł	SPI	15
Tranceiver CAN	TJA1050 moduł	CAN H/L	8
Rezystor terminacyjny	120 Ω 0.25W	włutowany	1
Złącze DB9 (gniazdo)	standard	śrubowy	5
Kabel CAN 2-wire 1m	skrętka	swobodny	10
Razem (wariant ESP32)			64
Razem (wariant PCAN)			664

15.2 Oprogramowanie — zależności (bezpłatne)

Tabela 25: Zestawienie zależności programowych — licencje

Biblioteka	Wersja	Licencja	Koszt
Qt6	6.6	LGPL v3 / GPL v3	bezpłatna (open source)
Google Test	1.14.0	BSD 3-Clause	bezpłatna
Lua	5.4.6	MIT	bezpłatna
OpenCL	1.2+	Khronos	bezpłatna
libzstd	1.5.5	BSD / GPL v2	bezpłatna
Mongoose	7.x	GPL v2 / Commercial	bezpłatna (GPL)
MSYS2 / GCC 13	—	GPL	bezpłatna
Łącznie			0 PLN

16 Struktura plików i konfiguracja

16.1 Pełna struktura katalogów projektu

Tabela 26: Szczegółowa organizacja katalogów projektu MagistralaCAN4

Katalog / plik	Pliki	Zawartość
src/core/	≈95	Logika biznesowa: parsery, silniki, sterowniki, widgety.
src/gui/	3	MainWindow, HeatmapBar, LazyTabWidget.
tests/	9	Testy jednostkowe (344 przypadki, Google Test).
esp_mcp/	1 (.ino)	Firmware ESP32+MCP2515 (Arduino IDE).
esp_slcan/	1 (.ino)	Firmware ESP32 SLCAN (protokół ASCII).
esp_gvret/	1 (.ino)	Firmware ESP32 GVRET (protokół binarny EVTV).
build_native/	1 (.exe)	Skompilowany plik wykonywalny Windows (standalone).
raprort/	1 (.tex)	Niniejsza dokumentacja projektu.
.github/workflows/	2 (.yaml)	Potok CI/CD: ci.yaml (Linux+ASAN), build.yaml (Windows MSYS2).
CMakeLists.txt	1	Konfiguracja budowania (CI-aware: ENV{CI} guard).
send_frame.lua	1	Przykładowy skrypt Lua.

16.2 Pliki konfiguracyjne i auto-save

Tabela 27: Pliki konfiguracyjne, sesji i auto-save aplikacji

Plik / katalog	Opis
%USERPROFILE%\ .magistrala_can4\ settings.ini	Ustawienia okna, motyw, ostatnie pliki DBC/Lua, tryb nadpisywania.
%USERPROFILE%\ .magistrala_can4\token	Token 256-bit serwera WSS (64 znaki hex).
%USERPROFILE%\ .magistrala_can4\server.crt	Certyfikat TLS self-signed (generowany automatycznie).
autosave_learner.json	Auto-save sesji uczenia asocjacyjnego (co 5 min).
magistrala_debug.log	Log zdarzeń aplikacji (klasa <code>Logger</code>).
C:\candump\candump_YYYYMMDD_HHMMSS.log	Automatyczne nagrania candump (ASCII).
*.mcan / *.mcan.zst	Nagrania binarne sesji CAN (opcjonalna kompresja zstd).
*.mf4	Nagrania MDF4 (AUTOSAR, kompatybilne z CANalyzer/PEAK).
*.pcap	Eksport (LINKTYPE_CAN_SOCKETCAN=227, Wire-shark).
	PCAP

17 Instrukcja użytkownika

17.1 Uruchomienie i pierwsze kroki

1. Podłączyć adapter CAN (PCAN-USB, Canable lub ESP32+MCP2515) do portu USB komputera.
2. Uruchomić `MagistralaCAN4.exe` (Windows) lub `./MagistralaCAN4` (Linux).
3. W grupie zakładek **Przechwytywanie** → **Ruch CAN**:
 - wybrać interfejs z listy (odświeżanej co 5 s);
 - ustawić prędkość magistrali (domyślnie 500K);
 - wcisnąć [**>**] **Start**.
4. Ramki CAN pojawiają się w tabeli w czasie rzeczywistym.

17.2 Podstawowe operacje

Tabela 28: Podstawowe operacje użytkownika

Operacja	Lokalizacja	Skrót / akcja
Start / Stop sniffingu	Pasek narzędzi	Przycisk [>] Start / [STOP] Stop
Filtrowanie po ID	CanStatsPanel	Pole tekstowe (hex lub wyrażenie)
Kopiowanie ramek	Tabela → prawy klik	Ctrl+C lub menu kontekstowe
Eksport candump	Pasek narzędzi	Eksportuj candump
Eksport CSV	Pasek narzędzi	Eksportuj CSV
Eksport PCAP	Pasek narzędzi	Eksportuj PCAP
Wczytaj plik DBC	Pasek narzędzi	Wczytaj DBC
Nagrywanie MCAN	Pasek narzędzi	Nagraj
Odtwarzanie nagrania	Pasek narzędzi	Wczytaj nagranie → Odtwórz
Uczenie asocjacyjne	Analiza → Uczenie	Ctrl+Shift+E (zdarzenie)
Uruchomienie ICSim	Narzędzia → ICSim	[>] Uruchom ICSim
Zdalny CAN (klient)	Narzędzia → Zdalny CAN	Wpisz URL ws:// lub wss://
Undo / Redo modelu	Klawiatura	Ctrl+Z / Ctrl+Y

17.3 Filtrowanie wyrażeniami

Pole filtra w CanStatsPanel akceptuje wyrażenia logiczne:

Listing 6: Przykłady filtrów wyrażeniowych

```

1 // Tylko ramki 0x7DF
2 7DF
3
4 // ID z zakresu diagnostyki UDS
5 can.id >= 0x700 && can.id <= 0x7FF
6
7 // Ramki z DLC = 8, nie-RTR
8 can.dlc == 8 && !can.rtr
9
10 // Wiele ID jednocześnie
11 can.id == 0x244 || can.id == 0x19B || can.id == 0x0AA

```

17.4 Integracja z ICSim

ICSim (Instrument Cluster Simulator) pozwala symulować CAN dashboardu pojazdu na wirtualnej magistrali vcan0. MagistralaCAN4 łączy się z symulatorem przez WebSocket bridge (magistrala_bridge.exe).

1. W zakładce **Narzędzia** → **ICSim** ustawić katalog **builddir** (domyślnie E:/ICSim/builddir).
2. Kliknąć [>] **Uruchom ICSim** — aplikacja automatycznie:
 - startuje `magistrala_bridge.exe -port 9001 vcan0`,

- startuje `icsim_imgui.exe vcan0` (symulator dashboardu),
- po 400 ms łączy klienta WebSocket z `ws://127.0.0.1:9001`.

3. Ramki z ICSim pojawiają się w zakładce *Ruch CAN* oraz w *Uczeniu asocjacyjnym*.

Identyfikatory CAN używane przez ICSim (plik `default.toml`):

Tabela 29: Identyfikatory CAN symulatora ICSim

CAN ID	Sygnal	Kodowanie
0x244	Prędkość	bajty 3–4, big-endian, $\div 100 \times 0.6214 = \text{mph}$
0x19B	Drzwi	bajt 2, bitmask: D1=0x01, D2=0x02, D3=0x04, D4=0x08
0x188	Kierunkowskazy	bajt 0: left=0x01, right=0x02
0x0AA	Obroty silnika	bajty 4–5, $\div 4 = \text{RPM}$
0x1B8	Temperatura	bajt 2 (wartość surowa)
0x2C8	Paliwo	bajt 5 (wartość surowa)

18 Instrukcja serwisowa

18.1 Budowanie ze źródeł

Listing 7: Budowanie projektu (Windows, MSYS2 UCRT64)

```

1 # Instalacja Qt6 statyczne (opcja dla dystrybucji standalone)
2 pacman -S mingw-w64-ucrt-x86_64-qt6-static
3
4 # Konfiguracja i kompilacja
5 cmake -B build_win -G "MinGW Makefiles" \
6     -DCMAKE_BUILD_TYPE=Release \
7     -DCMAKE_PREFIX_PATH="C:/msys64/ucrt64/qt6-static"
8 cmake --build build_win --target MagistralaCAN4 -j8

```

18.2 Uruchamianie testów

Listing 8: Uruchomienie testów jednostkowych

```

1 cmake --build build_win --target MagistralaCAN4Test -j4
2 cd build_win && ctest --output-on-failure
3 # Raport pokrycia (Linux, wymaga gcov + lcov):
4 lcov --capture --directory . --output-file coverage.info
5 genhtml coverage.info --output-directory coverage_html

```

18.3 Konfiguracja wirtualnej magistrali CAN (Linux)

Na systemach Linux można testować aplikację bez fizycznego sprzętu używając wirtualnego interfejsu `vcan0`:

Listing 9: Konfiguracja `vcan0` i generowanie ramek testowych

```
1 # Zaladuj modul vcan
2 sudo modprobe vcan
3 sudo ip link add dev vcan0 type vcan
4 sudo ip link set up vcan0
5
6 # Generuj losowe ramki (can-utils)
7 cangen vcan0 -v -I 123 -L 8 -g 10
8
9 # Generuj ramki CANopen (testy protokolu)
10 cangen vcan0 -I 0x701 -L 1 -D 05 -g 100 # Heartbeat Operational
11 cangen vcan0 -I 0x000 -L 2 -D 01 00 -g 1000 # NMT Start
```

18.4 Regeneracja tokena WSS

Token 256-bit serwera WSS jest przechowywany w pliku %USERPROFILE%\
.magistrala_can4\

token. Aby wygenerować nowy token (np. po kompromitacji):

Listing 10: Regeneracja tokena WSS

```
1 # Linux / MSYS2:
2 rm ~/.magistrala_can4/token
3 # Nastepne uruchomienie serwera WSS wygeneruje nowy token.
4 # Wszyscy klienci musza otrzymac nowy token.
```

18.5 Możliwe przyczyny awarii i diagnoza

Tabela 30: Diagnoza najczęstszych problemów

Objaw	Przyczyna	Rozwiązanie
Brak interfejsów na liście	Brak sterownika PCAN lub zainstalowany interfejs COM	Zainstalować PCAN-Basic API (Peak Systems) lub sprawdzić Menedżer urządzeń
„Brak ramek po 5s” (ostrzeżenie)	Zły baud rate lub brak aktywności na magistrali	Sprawdzić prędkość magistrali; podłączyć co najmniej 2 węzły CAN
ICSim nie uruchamia się	Brak <code>magistrala_bridge.exe</code>	Skompilować ICSim: <code>cd E:\ICSim && meson compile -C builddir</code>
WebSocket nie łączy (wss://)	Błąd certyfikatu SSL	Sprawdzić URL i token; aplikacja ignoruje self-signed cert
Tabela nie odświeża się (zdalny CAN)	<code>m_batchTimer</code> zatrzymany (stara wersja)	Zaktualizować do wersji z <code>m_batchTimer.start()</code> w konstruktorze
Candump pusty (zdalny CAN)	<code>m_candumpStream</code> = nullptr bez sniffingu	Zaktualizować do wersji z leniwym otwarciem w <code>onNewFrame()</code>
Segfault przy dużym ruchu	Przepelnienie <code>m_frameBuffer</code>	Limit wynosi 10 000 ramek; przy wyższym ruchu włączyć tryb nadpisywania

18.6 Pliki konfiguracyjne i logi

Tabela 31: Pliki konfiguracyjne i logi aplikacji

Plik / katalog	Opis
C:\candump\candump_YYYYMMDD_HHMMSS.log	Automatyczne nagrania candump (format ASCII)
%USERPROFILE%\magistrala_can4\settings.ini	Ustawienia okna, motyw, ostatnie pliki DBC/Lua
%USERPROFILE%\magistrala_can4\filter_presets.txt	Zapisane preset-y filtrów (format: hexID nazwa)
magistrala_debug.log	Log zdarzeń w katalogu roboczym (klasa Logger)
*.mcan / *.mcan.zst	Nagrania binarne sesji CAN (opcjonalna kompresja zstd)
*.mf4	Nagrania w formacie MDF4 (AUTOSAR, kompatybilne z CANalyzer)

19 Wykaz skrótów i pojęć

Tabela 32: Wykaz skrótów i pojęć używanych w dokumentacji

Skrót / pojęcie	Rozwinięcie i znaczenie
CAN	Controller Area Network — przemysłowa sieć szeregową do komunikacji ECU.
CAN FD	CAN with Flexible Data-Rate — ramki do 64 B, prędkość do 8 Mbit/s.
CAN XL	CAN XL — ramki do 2048 B, prędkość do 20 Mbit/s (ISO 11898-1:2024).
ECU	Electronic Control Unit — elektroniczny sterownik pojazdu.
UDS	Unified Diagnostic Services — protokół diagnostyczny ISO 14229.
ISO-TP	ISO 15765-2 — protokół transportowy CAN (segmentacja do 4095 B).
J1939	SAE J1939 — protokół dla pojazdów ciężarowych (PGN/SPN).
OBD-II	On-Board Diagnostics II — standard diagnostyki emisji (SAE J1979).
CANopen	CiA 301 — protokół aplikacyjny dla automatyki przemysłowej.
SOME/IP	Scalable service-Oriented MiddlewarE over IP — AUTOSAR middle-ware.
DoIP	Diagnostics over IP — ISO 13400, diagnostyka przez Ethernet.
KWP2000	Keyword Protocol 2000 — ISO 14230, poprzednik UDS.
XCP	Universal Measurement and Calibration Protocol — ASAM.
LIN	Local Interconnect Network — jednokierunkowa sieć 20 kbit/s.
PCAN	Peak CAN — interfejs USB-CAN firmy Peak Systems.
SLCAN	Serial Line CAN — protokół ASCII (Lawicel) dla adapterów USB-CAN.
GVRET	GVRET (EVTV Motor Werks) — binarny protokół sterownika SavvyCAN.
MCP2515	Kontroler CAN firmy Microchip — połączony z ESP32 przez SPI.
DBC	CAN Database Container — format pliku z definicjami sygnałów.
ARXML	AUTOSAR XML — format opisujący sieci CAN w standardzie AUTOSAR.
MDF4	Measurement Data Format 4 — ASAM, format nagrań (CANalyzer).
MCAN	Format nagrań własny MagistralaCAN4 (<code>.mcan</code> / <code>.mcan.zst</code>).
PCAP	Packet CAPture — format Wireshark (<code>LINKTYPE_CAN_SOCKETCAN=227</code>).
WSS	WebSocket Secure — WebSocket z szyfrowaniem TLS 1.3.
MQTT	Message Queuing Telemetry Transport — protokół IoT (pub/sub).
ICSim	Instrument Cluster Simulator — symulator deski rozdzielczej.
GUI	Graphical User Interface — graficzny interfejs użytkownika.
ML	Machine Learning — uczenie maszynowe.
EWMA	Exponentially Weighted Moving Average — wykładnicza średnia ważona.
MIC	Maximal Information Coefficient — miara zależności statystycznej.
PCA	Principal Component Analysis — analiza składowych głównych.
WCSS	Within-Cluster Sum of Squares — kryterium doboru k w k -means.
CI/CD	Continuous Integration / Continuous Deployment — automatyzacja budowania.
ASAN	AddressSanitizer — narzędzie wykrywania błędów pamięci (GCC/Clang).

20 Uwagi dodatkowe

20.1 Historia wersji

Tabela 33: Historia wersji projektu MagistralaCAN4

Wersja	Data	Główne zmiany
1.0	2024 Q1	Pierwsze wydanie: CanSniffer, PCAN, SLCAN, CanFrame-Model, podstawowy GUI Qt6.
1.5	2024 Q2	AssociativeLearner (v1), LuaScriptEngine, DbcParser, eksport candump/CSV.
1.8	2024 Q3	J1939Widget, UdsWidget + ISO-TP, OBD-II, CANopen, DBC edytor, HTTP REST API.
1.9	2024 Q4	WebSocket serwer (port 9000), RemoteCanWidget (ws:// + wss:// + TLS token).
2.0	2025 Q1	MqttBridge, MDF4Writer, EspMcpDriver, GVRET binarny, CanRecorder (.mcan).
2.0.5	2025 Q2	IcSimWidget + magistrala_bridge, SOME/IP, DoIP, LIN, KWP2000, XCP parsery.
2.1.0	2025 Q3	LearningEngine v2.1 (37 algorytmów), CanBusHealthWidget (AUTOSAR counter), CanTriggerWidget, CanSignal-StatisticsWidget, 344 testy jednostkowe, CI/CD (GitHub Actions), CAN XL, SQLite WAL, PCAP eksport.

20.2 Plany rozwoju

- **Wsparcie PyTorch/libtorch** — integracja silnika uczenia z modelem PyTorch przez C++ API (libtorch), umożliwiająca trenowanie sieci neuronowych na danych CAN bezpośrednio w aplikacji.
- **Panel webowy** — rozszerzenie REST API (:8080) o serwowanie interaktywnego widoku tabeli w przeglądarce; infrastruktura `HttpRestServer` jest już gotowa.
- **Wtyczki dynamiczne** — rozbudowa systemu `PluginLoader` o GUI konfiguracji i hot-reload `.dll/.so` z katalogu `./plugins`.
- **Pełna obsługa AUTOSAR ARXML** — kompletny import i eksport artefaktów `.arxml` z mapowaniem sygnałów; częściowo zrealizowany przez `ArxmlParser`.
- **Tryb multi-bus** — jednoczesne przechwytywanie z wielu interfejsów CAN w jednej sesji (np. PCAN + SLCAN równolegle).
- **Export do ASC (CANalyzer)** — format `.asc` Vector obok istniejących `.mcan`, `.mf4` i `candump`.

20.3 Podsumowanie

MagistralaCAN4 jest kompletnym środowiskiem diagnostycznym dla magistrali CAN, gotowym zarówno do użytku laboratoryjnego (analiza ECU, diagnostyka UDS), jak i edukacyjnego (symulator ICSim, uczenie asocjacyjne bez etykiet). Modułowa architektura Qt i podział na warstwy (sterownik → sniffer → model → GUI) ułatwia dodawanie nowych interfejsów i protokołów bez ingerencji w istniejący kod.

Kluczowe dane techniczne:

Tabela 34: Parametry techniczne MagistralaCAN4 v2.1.0

Parametr	Wartość
Wersja	2.1.0
Język programowania	C++17
Framework GUI	Qt 6.6
System budowania	CMake 3.16+
Liczba plików źródłowych	>160
Liczba widgetów	≈40
Obsługiwane protokoły warstwy 7	UDS, J1939, OBD-II, CANopen, SOME/IP, DoIP, KWP2000, XCP, LIN
Interfejsy sprzętowe	PCAN, SLCAN, ESP32-MCP2515, GVRET
Interfejsy sieciowe	WebSocket (<code>ws://</code> oraz <code>wss://</code>)
Format nagrań	.mcan, .mcan.zst, .mf4, .pcap, candump, CSV
Testy jednostkowe	344 przypadki, 9 plików (Google Test)
Licencja	Prywatna (zależności: LGPL/MIT/BSD/GPL)